The Geometry and Transfer of RLVR in Small Reasoning Models

Sunny Guha[*] *email TBD*
Oracle

Chaitanya Afle[*] *email TBD*
Cranium.ai

Abstract

Reinforcement Learning with Verifiable Rewards (RLVR) is now the default recipe for eliciting reasoning in language models, yet its training dynamics in *small* reasoning models remain comparatively under-studied; we use GRPO across ten `reasoning-gym` tasks to ask two questions: how RLVR *transfers* across tasks, and what it does to a model's *weight-space geometry*. **Transfer.** RLVR is not a universal improver. Across a 10×10 cross-task transfer matrix evaluated at four model scales, GRPO degrades 36% of (train, eval) task pairs, with catastrophic cross-task drops of up to -52 points. Learning rate, not model size, is the controlling hyperparameter, and arithmetic degrades at every scale. **When RLVR helps.** To predict *when* RLVR helps, we introduce the *Atomic Readiness Score* (ARS), a $\sim$1-GPU-hour zero-shot probe that decomposes a task into grounding (L1) and local-reasoning (L2) atoms. From these atoms, ARS recommends one of three training regimes (Direct, Atomic-Only, Atomic-Warm) before any GRPO run. It selects the optimal regime on 6/7 tasks, and its single failure is mechanistically diagnostic rather than random. **Geometry.** The task vector $\tau = W_{\text{GRPO}} - W_{\text{SFT}}$ carries a scale- and family-invariant signature: GRPO preserves W's spectrum, τ is near-orthogonal to W's top subspace, and the `mlp.down_proj` basis is *frozen* (0° rotation) while `mlp.up_proj`/`attn.q_proj` rotate 1.5°–2.7°, growing monotonically with scale. The rotation *magnitude* is identical for every task (per-task spread $\leq 0.06^\circ$): GRPO spends a fixed per-layer rotation budget regardless of what it learns, and tasks differ only in the *direction* of that rotation and the network *depth* at which each writes its change, with the depth shift across scale itself task- and family-dependent. This geometry delivers one universally safe task-arithmetic merging recipe across 19 pairs (discard `mlp.down_proj` contributions: 40/48 cells lossless), shows simple summation beats $1/N$ averaging, and that atomic- and direct-trained vectors compose in distinct but cross-scale-stable ways. It does not, however, buy prospective predictability: two merge predictors strong at n=7 (weight-space sign-agreement r=-0.81, behavioral L1-grounding r=-0.76) both collapse at n=19 (-0.27 and $+0.06$), and merge-compatibility is *pair-dependent*, not task-intrinsic. We argue these honest negatives are a contribution: the field has implicitly assumed task-intrinsic merge structure and cheap pre-probes, and our evidence rules out both for reasoning RLVR vectors. We release all checkpoints, probes, and analysis code.

1 Introduction

Reinforcement Learning with Verifiable Rewards (RLVR), typically Group Relative Policy Optimization (Shao et al., 2024) with a binary correctness reward, has become the standard tool for turning a pretrained language model into a reasoner (DeepSeek-AI, 2025). Most of the public understanding of RLVR comes from *large* models, where it reliably helps. For *small* models (here $\leq$ 8B), the picture is murkier, and practitioners often treat GRPO as a monotone improvement operator: run it on a task, get a better model. This paper

[*]Work done outside the affiliated institutions.

is a systematic, multi-phase rebuttal of that assumption, and an attempt to replace it with a mechanistic account of what GRPO actually *does* to a small model's weights and capabilities.

We organize the study as four interlocking phases, each answering a distinct question and feeding the next:

1. **Does single-task GRPO transfer (Phase 1)?** We build full 10×10 transfer matrices at 3B/4B/7B/8B. GRPO helps some tasks dramatically (countdown +60 at 7B) but *degrades* 36% of (train, eval) pairs, sometimes catastrophically (-52). Learning rate, not scale, is the dominant control.
2. **Can a cheap probe predict the right RLVR regime (Phase 2)?** We decompose each task into atomic sub-skills and define the Atomic Readiness Score (ARS). ARS predicts the best of {Direct-RLVR, Atomic-Only, Atomic-Warm} on 6/7 tasks, and the one miss (`mini_sudoku`) pinpoints *why* the framework can fail.
3. **What does GRPO do to the weights (Phase 3)?** We characterize the task vector $\tau = W_{\text{GRPO}} - W_{\text{SFT}}$ geometrically. GRPO has a universal structural signature (spectrum-preserving, `mlp.down_proj`-basis-frozen, feature-projector rotations of $1.5°$–$2.7°$) whose *magnitude* is the same for every task, a fixed per-layer rotation budget, and a task-specific *location*: which depths each task rotates and in which direction, with the depth shifting task- and family-dependently across scale.
4. **Can task vectors be merged into multi-skill models (Phase 4)?** The Phase-3 geometry motivates task-arithmetic merging. We find one universally safe recipe and a clean training-regime effect, but we also show, on an expanded n=19 sample, that every cheap *prospective* predictor of merge damage we tried (including two we initially believed at n=7) fails, and that merge compatibility is a property of the *pair*, not the task.

Contributions. (i) The first large cross-scale RLVR transfer matrix for small models, establishing that GRPO is not free improvement and that LR dominates scale (§4). (ii) ARS, a ∼1-hour zero-shot diagnostic that selects the RLVR regime, validated on 6/7 tasks, plus craft lessons (long-answer atoms, single hard difficulty transitions, the harm of partial credit) from a 15-condition shortest-path study (§5). (iii) A scale- and family-invariant geometric signature of GRPO updates, including the `mlp.down_proj`-frozen invariant and a fixed per-layer rotation budget that every task spends identically (verified per task) and redirects only in rotation direction and network depth (§6). (iv) A practical, geometrically motivated merge recipe; a cross-scale-stable atomic-vs-direct composition effect; and a rigorously documented *null-predictor catalogue* that corrects two of our own earlier (n=7) claims (§7). We view (iv)'s honest negatives, surviving expansion to n=19, as a first-class contribution to a literature that has assumed the opposite.

2 Related Work

RLVR and GRPO. GRPO (Shao et al., 2024) replaces PPO's (Schulman et al., 2017) learned value function with a group-relative advantage, and underlies recent reasoning systems (DeepSeek-AI, 2025). DAPO (Yu et al., 2025) refines the loss (decoupled clipping, dynamic sampling); we use the DAPO variant with β=0 throughout. Prior work studies RLVR at scale; we instead ask what it does *mechanistically* to small models, where its failure modes are visible.

Curriculum and decomposition. Curriculum learning (Bengio et al., 2009) orders examples of the *same* task by difficulty; least-to-most and chain-of-thought prompting (Zhou et al., 2023; Wei et al., 2022) decompose at inference time with no training signal. Our atomic curriculum (Phase 2) instead *trains* on categorically distinct prerequisite sub-skills with verifiable rewards, and uses a pre-training probe (ARS) to decide *whether* decomposition will help.

Task arithmetic and model merging. Editing models with task arithmetic (Ilharco et al., 2023), TIES-Merging (Yadav et al., 2023), DARE (Yu et al., 2024), model soups (Wortsman et al., 2022), and Fisher-weighted averaging (Matena & Raffel, 2022) merge fine-tuned checkpoints by combining weight deltas.

This literature largely studies supervised/instruction vectors and assumes that merge compatibility can be predicted or repaired by sparsification/sign-resolution heuristics. Phase 4 stress-tests these assumptions specifically for *RLVR* vectors on reasoning tasks and finds the prospective-prediction premise unsupported at n=19.

Geometry of fine-tuning. Low intrinsic dimensionality (Aghajanyan et al., 2021), LoRA (Hu et al., 2022), and orthogonal fine-tuning (Qiu et al., 2023) argue that adaptation lives in a small or rotation-like subspace. Phase 3 shows GRPO exhibits this pattern unusually strongly (spectrum-preserving, near-orthogonal τ) and localizes it to specific layer types, which directly informs the merge recipe of Phase 4.

3 Experimental Setup

Models. We study four post-SFT checkpoints spanning two Qwen families and four scales (Table 1). The Qwen2.5/Qwen3 split (3B/7B vs. 4B/8B) is a family confound we cannot fully disentangle from scale and flag throughout. Before any RL, each base is SFT'd for 3 epochs (lr=2×10^{-5}) on 1,000 *task-agnostic* warm-up samples (trivia, unit conversions, word puzzles, date reasoning) to teach the structured `<think>...</think><answer>...</answer>` contract; the data is deliberately task-agnostic so no `reasoning-gym` task gains an advantage. These W_{SFT} checkpoints are the reference point throughout.

Table 1: The four models. Qwen3 has no separate `-Instruct` variant; its base already follows instructions.

Scale	Family	Params	HuggingFace ID
3B	Qwen2.5	3B	`Qwen/Qwen2.5-3B-Instruct`
4B	Qwen3	4B	`Qwen/Qwen3-4B`
7B	Qwen2.5	7B	`Qwen/Qwen2.5-7B-Instruct`
8B	Qwen3	8B	`Qwen/Qwen3-8B`

Tasks. We use ten `reasoning-gym` (Reasoning Gym Contributors, 2025) tasks spanning five skill categories (Table 2). Each task has a programmatic verifier returning a binary reward.

Table 2: The ten `reasoning-gym` tasks, by skill category.

Category	Task	Description
Arithmetic	`basic_arithmetic`	Evaluate arithmetic expressions (2–6 terms, 1–4 digits)
	`countdown`	Reach a target number using a given set of numbers
Logic	`syllogism`	Determine the validity of a syllogistic argument
	`knights_knaves`	Infer who is a knight/knave from their statements
Spatial/Grid	`maze`	Find the shortest-path length in an ASCII maze
	`mini_sudoku`	Solve a 4×4 Sudoku puzzle
	`shortest_path`	Find the shortest-path directions in a grid
Planning	`leg_counting`	Count the total legs for a list of animals
Algorithmic	`caesar_cipher`	Decrypt a Caesar cipher (rotation 1–5)
	`cryptarithm`	Solve a letter-to-digit arithmetic puzzle

GRPO recipe. Unless noted: DAPO loss, lr=5×10^{-6} constant, 300 steps (single-task), 8 generations/prompt, β=0, no KL penalty, temperature 1.0, `mask_truncated_completions`, ϵ_{high}=5.0, `stop_strings` = `</answer>`. A learning-rate sweep (1×10^{-6}, 16 generations) is reported for 7B/8B in Phase 1. Trained with TRL (von Werra et al., 2020) under 8-GPU DDP.

Evaluation. `evaluate_transfer.py`, 200 samples/task (50 in Phase 1), seed 9999 (distinct from training seed 42), greedy-free sampling, last-`<answer>` extraction. An early Phase-1 verifier bug (first-match `<answer>` caught an echoed template tag) was fixed to last-match; all Phase 2–4 numbers are post-fix.

4 The Cross-Task Transfer Matrix

A natural hope for RLVR is that training a model on one reasoning task makes it a better reasoner in general. We test this directly. For each of the ten tasks, at each of the four scales, we train a separate single-task GRPO model and then evaluate it on all ten tasks, yielding a 10×10 transfer matrix per scale. Each cell is the change in accuracy relative to that model's SFT starting point (positive = GRPO improved the task, negative = it degraded the task); the diagonal measures same-task learning, the off-diagonal measures cross-task transfer. Throughout we focus on the eight non-trivial tasks (`caesar_cipher` and `cryptarithm` sit at 0% throughout and are omitted), and Figure 1 renders all four scales as heatmaps on a shared color scale; the full 7B/8B matrices and the learning-rate sweep are in Appendix A.

GRPO is not a universal improver. The headline result is that GRPO is not free improvement: it helps on some tasks, hurts on others, and across all four matrices it degrades 36% of (train, eval) task pairs. The damage can be catastrophic and crosses unrelated skills, training on the constraint-satisfaction task `mini_sudoku` at 4B costs −52 pts on `leg_counting` and −50 on `knights_knaves`. GRPO can even hurt the very task it was trained on: `basic_arithmetic` degrades on *itself* at every scale (−14 at 3B, −12 at 4B, −30 at 7B). Its binary reward rewards short final answers and induces a *reasoning compression* that skips the intermediate steps multi-digit arithmetic needs, so the verbose chain-of-thought learned during SFT is a better strategy than anything GRPO discovers. Large gains coexist with all of this (`countdown` reaches +60 at 7B), but the net effect is task- and scale-dependent, not a uniform lift.

What consistently helps, what stays inert. Three tasks improve at *every* scale and share a common trait: they have clear, learnable reasoning structure that GRPO can sharpen without needing novel computation. `syllogism` gains +18–+26, `shortest_path` +28–+40, and `countdown` scales with model size (+10 → +36 → +60 → +50 for 3B/4B/7B/8B). At the other extreme, `caesar_cipher` and `cryptarithm` sit at 0% baseline *and* 0% after GRPO at all scales: their reward is too sparse (a single wrong character fails the whole answer) to give any learning signal. Between these lies a capability threshold: `mini_sudoku` is unlearnable below 4B but reaches +60 at 8B, once the model has the capacity to track row, column, and subgrid constraints simultaneously.

Transfer is asymmetric and structured. Cross-task transfer is not symmetric. Search and planning training transfers positively to logic (`countdown`→ `syllogism` is +10–+18; `shortest_path`→`syllogism` is +16 at 4B), but logic training hurts computation (`syllogism`→ `basic_arithmetic` is −20 at 3B and −46 at 7B). `shortest_path` is the best all-rounder, lifting 6 of 9 other tasks at 4B (up to +28 on `mini_sudoku`), whereas training on `mini_sudoku` overspecializes the model and is the most destructive to others. This directional structure is what makes curriculum order matter for multi-task RLVR.

Baselines are strong, and learning rate dominates scale. SFT alone already reaches 62–78% on arithmetic and syllogism, so GRPO's room to improve over SFT is smaller than is commonly assumed, and its room to *damage* a strong baseline is correspondingly larger. On top of this, the learning rate is the dominant hyperparameter. Re-running 7B/8B at the gentle lr=1×10^{-6} (16 generations) collapses both gains and harms: the worst 7B same-task drop shrinks from −30 to −4 and the largest gain from +60 to +8, flattening the entire matrix to within ±18 pts (Figure 1 bottom row, Appendix A). The difference between 5×10^{-6} and 1×10^{-6} exceeds the difference between model sizes; the optimal LR sits between, high enough to learn but low enough to avoid forgetting. These findings motivate the rest of the paper: *when* does GRPO help (§5), *what* does it change (§6), and can those changes be *composed* (§7)?

Figure 1: **Cross-task GRPO transfer at all four scales**, on a shared ±52 color scale. Rows are the training task, columns the evaluation task; cells are Δ accuracy vs. the SFT baseline (red = gain, blue = loss); the bold diagonal is same-task; the bottom row of each panel gives the absolute SFT baseline accuracy per task. *Top row* (3B, 4B) uses the aggressive lr=5×10^{-6}: the catastrophic 4B `mini_sudoku` row (−50, −52) and the asymmetric search→logic / logic→arithmetic structure are visible at a glance. *Bottom row* (7B, 8B) uses the gentle lr=1×10^{-6} (v2): on the same scale the entire matrix flattens to within a few points (no cell exceeds ±18), eliminating both the catastrophic harms and the large gains seen above. The lone systematic exception is the negative `kk` column at 8B (GRPO on any task degrades its fragile 74% `knights_knaves` baseline).

5 Atomic Readiness and the Curriculum Decision

The transfer matrix (§4) showed that single-task GRPO does not uniformly help, and that on some tasks it actively destroys a working model (`maze` collapses to 1%). We now ask *when* GRPO will help, and turn the answer into a cheap pre-training diagnostic.

Why decompose into atoms. Our starting hypothesis is about capability, not optimization. A hard reasoning task sits above the small model's current capability frontier: the model cannot reliably perform even the sub-steps the task is built from, so when we run GRPO directly on the full task it almost never stumbles onto a correct trajectory. Every sampled completion then earns near-zero verifiable reward, the group-relative advantage GRPO learns from is empty, and there is simply nothing to reinforce. Training

Table 3: **The three-layer task taxonomy, with example atoms.** L1 and L2 are short, programmatically verifiable Q&A atoms we author and train directly; L3 is the full task that only end-to-end GRPO trains. Examples are drawn from `maze`, `basic_arithmetic`, and `syllogism`.

Layer	What it tests	Representative atoms (task)
L1 grounding	parse the input	"What symbol is at row 3, column 5?" (`maze`); "List all numbers in $-66 - -51$" (`arithmetic`); "Which premise contains the word 'all'?" (`syllogism`)
L2 local reasoning	one inference step	"Is moving east from cell $(3, 5)$ legal?" (`maze`); "Evaluate $-(52 + -20 + -66)$" (`arithmetic`); "If A→B and A holds, what follows?" (`syllogism`)
L3 global planning	multi-step composition	give the shortest path from start to goal (`maze`); evaluate the full multi-operator expression under order-of-operations (`arithmetic`); decide whether the syllogism is valid (`syllogism`)

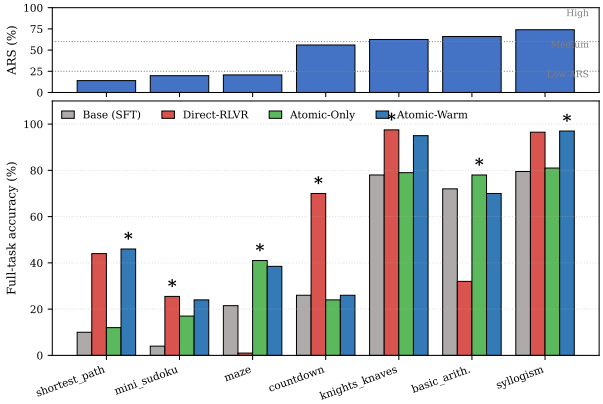
Table 4: **ARS predicts the optimal RLVR regime on 6/7 tasks.** Accuracies are full-task eval. Bold = best regime. ARS bands: high (> 60%), medium (40–60%), low (< 25%).

Task	ARS	Base	Direct	Atomic-Only	Atomic-Warm	ARS reading
syllogism	74%	79.5	96.5	81.0	**97.0**	high → GRPO works ✓
knights_knaves	62.5%	78.0	**97.5**	79.0	95.0	high → GRPO works ✓
basic_arithmetic	66%	72.0	32.0	**78.0**	70.0	high, but GRPO destructive ✓
countdown	56%	26.0	∼**70**	24.0	26.0	medium, L3 bottleneck ✓
maze	20.7%	21.5	1.0	**41.0**	38.5	low → atoms essential ✓
shortest_path	14%	10.0	44.0	12.0	**46.0**	low → atoms help ✓
mini_sudoku	19.8%	4.0	**25.5**	17.0	24.0	low, but GRPO wins ✗

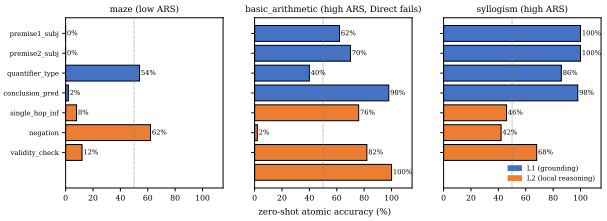
the full task harder cannot fix this. The alternative is to *grow the capability up to the task*: decompose it into easier atomic sub-skills that are already within reach (or nearly so), train those first to extend what the model can do, and only then expect the full task to become reachable. Under this view atoms are a curriculum that raises the model's floor until the full-task reward is no longer sparse, giving us a path to capabilities that direct training cannot reach.

A three-layer taxonomy. To make "sub-skill" precise we decompose any reasoning task into three layers (Table 3): **L1 (grounding)** reads the input representation, **L2 (local reasoning)** performs a single inference step over grounded elements, and **L3 (global planning)** is the multi-step composition that *is* the full task. L1 and L2 are the sub-skills we can author cheap atoms for and train in isolation; L3 is what full-task GRPO has to supply. The hypothesis above becomes concrete: when a model cannot do L1/L2, full-task GRPO has no foothold and thrashes on sparse reward, and an atomic curriculum that trains the missing layer should help.

The Atomic Readiness Score. For each task we hand-author 6–14 atomic Q&A *templates* (4–8 L1, 3–6 L2), each a deterministic function of a `reasoning-gym` sample with a programmatically verifiable answer. ARS is a pure inference probe: zero-shot the W_{SFT} base on 50 samples/atom, extract the last `<answer>`, score with a *lenient* type-aware verifier, and average. We report L1_avg (grounding floor), L2_avg (local-reasoning floor), and ARS (overall). The full 7-task probe costs ≈1 GPU-hour. We then compare three matched-compute RLVR regimes: **Direct-RLVR** (300 full-task steps), **Atomic-Only** (100 L1 → 100 L2, zero-shot transfer to the full task), and **Atomic-Warm** (L1 → L2 → full).



(a) Best regime by ARS band.



(b) L1/L2 atom accuracies per task.

Figure 2: **Atomic readiness.** (a) ARS partitions tasks into regimes; (b) the L1/L2 decomposition exposes which floor (grounding vs. local reasoning) is missing.

Main finding. ARS alone selects the optimal RLVR regime on 6 of 7 tasks (Table 4, Figure 2): the band a task falls in (high $> 60\%$, medium 40–60%, low $< 25\%$) tells us *in advance* whether to train the full task directly or to build an atomic curriculum first. We read the three bands in turn.

Where the curriculum works, and why. The clearest wins are the low-ARS tasks, exactly the regime our capability-frontier hypothesis predicts. `maze` (ARS 20.7%) cannot even locate start and goal, so Direct-RLVR collapses to 1%, *below* its 21.5% base: GRPO never samples a correct trajectory and so has nothing to reinforce. Teaching the grounding atoms first lifts it to 41% with no full-task training at all. `shortest_path` (ARS 14%) is the same story, atoms take it from 10% to 46% (55.5% with the refinements of Appendix D) where Direct-RLVR stalls at 44%. The mechanism is the one we hypothesized: when the L1/L2 floor is at zero the full-task reward is too sparse for GRPO to climb, and the only way forward is to *raise the floor* by training the missing sub-skills until the task comes within reach. The high-ARS tasks confirm the converse half of the same prediction: `syllogism` and `knights_knaves` already clear both floors, so Direct-RLVR reaches $\approx 97\%$ and atoms add almost nothing, ARS correctly tells us *not* to spend the extra compute.

The middle case. Medium-ARS `countdown` (56%) is the instructive in-between. Its floors are split (weak L1 number-handling, strong L2 arithmetic), but the real bottleneck is neither: it is the L3 search for a combination that hits the target, which no atom teaches. Accordingly both atomic regimes stay at base (24–26%) while only Direct-RLVR moves it ($\sim$70%). This is consistent with the hypothesis rather than a failure of it: atoms help only when the missing capability lives at L1 or L2, and when the gap is purely L3 planning the full task is the only thing that can supply it.

Where it does not work, and why. The lone misprediction is `mini_sudoku`: low ARS (19.8%) yet Direct-RLVR (25.5%) beats both atomic regimes. This is the honest boundary of the method, not a counterexample to the mechanism. Sudoku's true bottleneck is multi-cell constraint propagation, a holistic L2–L3 operation, but our single-hop atoms only probe isolated cell reads, so the ARS we measured is uncorrelated with the actual difficulty and the curriculum ends up training the wrong skill. The lesson is a clean one: *ARS is only as predictive as the atoms are well chosen*; when the atoms miss the task's real bottleneck, so does the score.

The arithmetic anomaly and reasoning compression. `basic_arithmetic` breaks the naive "high ARS $\Rightarrow$ GRPO works" rule: despite ARS=66%, Direct-RLVR collapses it to 32%. The cause is reasoning compression: GRPO shortens outputs (Atomic-Only 386 chars $\rightarrow$ Atomic-Warm 234 chars), skipping the order-of-operations steps (the `order_of_operations` atom sits at just 2% on the base model, the single weakest atom in the arithmetic decomposition). Generation logs at 8B make the mechanism visible directly: the SFT model writes out each step ("Subtracting a negative is like adding the positive…", $\approx$ 200 chars/response),

whereas the GRPO model collapses to a terse "$-66 - -51 = -15$" (≈ 70 chars). The short form is correct on one-step problems but fails on multi-step expressions like `-3241 - 2720 - 9730 + 6743 + 1933` where the intermediate values must be carried. ARS measures grounding readiness, not GRPO's reward dynamics; tasks with long obligatory reasoning chains can be harmed even at high ARS.

Craft lessons from pushing the hardest task. `shortest_path` is the lowest-ARS task in the suite (ARS 14%, base 10%), so we used it as a stress test for how far an atomic curriculum can be pushed. Across 16 controlled training conditions (Figure 3; full table in Appendix D) we took it from 10% to 55.5%, a 5.5× improvement and +11.5 points over plain Direct-RLVR. The conditions that helped, and the ones that backfired, distill into four reusable lessons for RLVR on hard tasks.

- **Match the atom's answer shape to the task.** Short yes/no grounding atoms train the model to emit one-token answers, the same reasoning compression we saw on arithmetic; transferred to a task that needs a written-out path this collapses to 12%, *below* the base. Replacing them with multi-step path-tracing atoms, whose answers are themselves short trajectories, removes the compression and lifts the otherwise-identical pipeline to 51.5%.

- **Prefer one hard jump over a gentle ramp.** A progressive curriculum ($3\times3 \rightarrow 4\times4 \rightarrow$ full grid) lets the model over-fit easy instances and plateau (27.5%); a single hard transition straight from atoms to full-size grids forces it to generalize and does markedly better.

- **Keep the reward binary.** Partial-credit rewards feel helpful but hurt GRPO (35.5%, and 24% when stacked with a progressive ramp). GRPO learns from the *variance* of rewards within a sampled group, so a smooth reward that gives every rollout a similar middling score flattens that variance and leaves the group-relative advantage with almost nothing to push on. The sparse, verifiable 0/1 reward gives the sharpest learning signal.

- **RFT-Base spends the whole budget on the real task.** The strongest recipe (55.5%) generates verified-correct traces from an already-trained model, supervised-fine-tunes the *base* model on them, and only then runs GRPO. Starting from a clean but already-capable initialization lets the entire GRPO step budget go to the full task, rather than being split between atoms and task.

A failure analysis of the 51.5% model (Appendix D) shows the residual errors are dominated by a "give-up reflex" (declaring no path exists when one does); removing that single behavior alone would reach 68%, pointing to where the next gains lie. None of these regimes harms general ability: an MMLU-math evaluation across 48 checkpoints spanning every part of the study finds no catastrophic forgetting (within ±3% of base; Appendix E).

Looking ahead: from growing one skill to composing many. The atomic curriculum shows how to bring a *single* hard skill within a small model's reach by raising its capability floor. The rest of the paper asks the complementary question: once several skills have been trained separately, can they be *combined* into one model without retraining, simply by adding their GRPO task vectors? We first characterize what GRPO does to the weights (§6) and find a geometry that looks ideal for composition, each task vector is tiny (0.06–0.12% of the weights), nearly orthogonal to the base, and nearly orthogonal to other tasks' vectors. We then test merging head-on (§7) and reach a split verdict that previews the paper's central tension: *construction is easy* (summing two task vectors yields a working multi-skill model, and one geometry-derived edit, dropping the never-moving `mlp.down_proj` block, is safe on 40/48 cells), but *prediction is hard* (no cheap probe says in advance which pairs merge cleanly; damage ranges from 0.5 to 64.5 points at near-identical global statistics). Tellingly, the readiness logic that works so well in this section does *not* carry over: neither ARS nor the sequential-transfer matrix of §4 predicts merge compatibility ($r \approx 0$), because merging is governed by a few localized per-parameter sign conflicts that every averaged metric washes out.

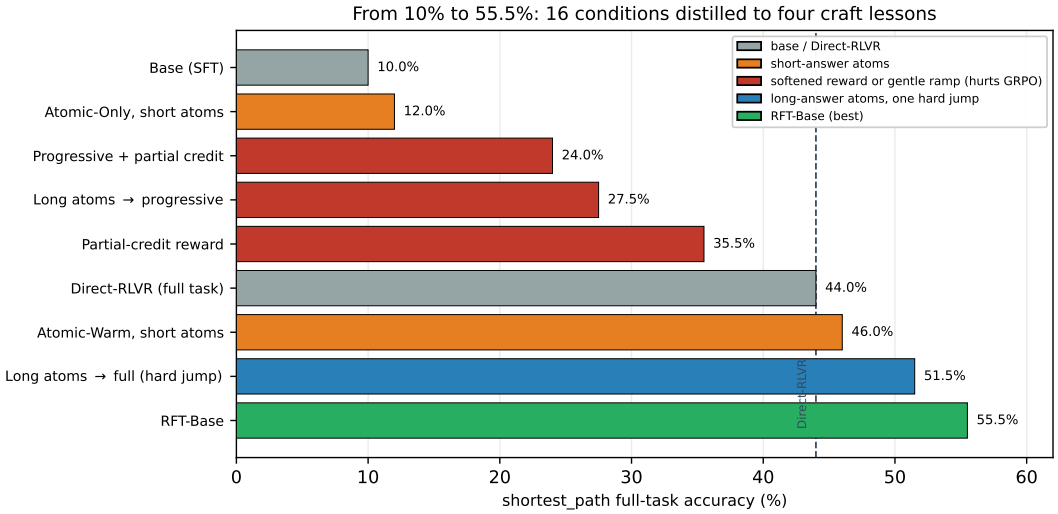


Figure 3: **Climbing the hardest task.** Sixteen training conditions take `shortest_path` from a 10% base to 55.5%. Bars are sorted by accuracy and colored by lesson category; the dashed line marks plain Direct-RLVR (44%), the bar to beat. Softened rewards and gentle difficulty ramps (red) fall *below* Direct-RLVR, short-answer atoms (orange) compress the output, and the wins come from long-answer atoms with one hard transition (blue) and RFT-Base (green).

6 The Weight-Space Geometry of GRPO

Having established *when* GRPO helps (§4–§5), we now open the model and ask what it physically does to the weights. For every (model scale, task) pair we form the **task vector**

$$\tau \;=\; W_{\mathrm{GRPO}} - W_{\mathrm{SFT}}$$

per weight matrix, the delta GRPO writes on top of the SFT base, and characterize it geometrically. We track six quantities, each isolating a distinct way a fine-tuning update can act on a weight matrix W (with singular value decomposition $W = U\Sigma V^\top$, singular values $\sigma_1 \geq \sigma_2 \geq \cdots$, and top-$k$ singular vectors U_k, V_k). Full definitions, estimators, and random-matrix baselines are in Appendix H.

- **Update magnitude** $\|\tau\|_F / \|W\|_F$: how big is the edit relative to the weight it edits?
- **Singular-value shift** $\sigma_i(W_{\mathrm{GRPO}})$ vs. $\sigma_i(W_{\mathrm{SFT}})$: does GRPO amplify or shrink the directions W already uses?
- **Effective rank** $\exp\big(H(\sigma/\textstyle\sum\sigma)\big)$ (entropy of the singular-value distribution): does GRPO concentrate or spread out W's information?
- **Subspace projection** $\rho_k(\tau) = \|U_k U_k^\top \tau\, V_k V_k^\top\|_F^2 / \|\tau\|_F^2$: what fraction of the update lands inside W's top-k directions, versus the random baseline $(k/r)^2$?
- **Basis rotation** $\arccos \sigma_i\big(U_{\mathrm{SFT}}^\top U_{\mathrm{GRPO}}\big)$ (principal angles): even with the σ_i fixed, do the singular *directions* swivel?
- **Depth signature** $\mathrm{depth}[\ell] = \sum_{\text{matrices at block } \ell} \|\tau\|_F$: where along the network's depth does a task concentrate its changes?

The picture splits cleanly in two. *How* GRPO edits a weight matrix is near-universal across tasks, scales, and model families; *where* it edits (which depths, and in which direction) is task-specific. We take the two halves in turn.

6.1 How GRPO edits: a universal, spectrum-preserving, rotation-localized signature

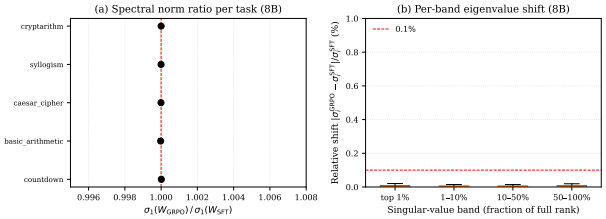
Every quantity in this subsection is computed *per (task, weight-matrix)* and reported as a mean over the tasks trained at each scale, grouped by layer type. The "universal" label is itself an empirical claim about the across-task spread: it is negligible (the per-layer-type rotation varies by $< 0.1°$ across tasks, and the $\pm$ figures below are across-task standard deviations; the full per-metric across-task spread is in Appendix H, Table 26), so the task-averaged numbers and the per-task numbers coincide, which is exactly why this half is universal and the "where" half below is not.

GRPO preserves the spectrum. The singular values σ_i of a weight matrix are the gains it applies along its principal directions; the largest, σ_1 (the spectral norm), is the biggest factor by which W can stretch any input vector, and the full ordered set $\sigma_1 \geq \sigma_2 \geq \cdots$ is the natural "size and shape" descriptor of a linear map. We compare these gains before and after training (Fig. 4a). On the most-changed layers the spectral-norm ratio $\sigma_1(W_{\text{GRPO}})/\sigma_1(W_{\text{SFT}})$ is 1.000 ± 0.002, the effective rank moves by less than 0.01, and a full-SVD sweep over twelve representative layers finds that *every* singular value shifts by less than 0.1%, at or below the bfloat16 rounding floor. **These statements hold at all four scales (3B, 4B, 7B, 8B) and across both model families** (Qwen2.5 at 3B/7B, Qwen3 at 4B/8B): the spectrum is preserved everywhere we looked. GRPO therefore neither amplifies, dampens, nor re-ranks the directions W already uses. Yet behavior demonstrably changes (§4–§5), so the entire learned change must live in directions that the SVD of W alone cannot see. The next two measurements locate them.

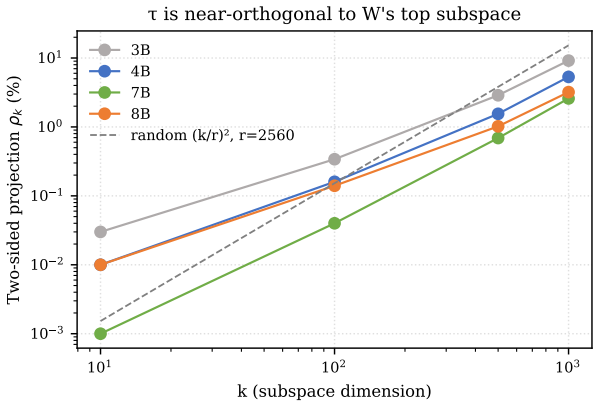
The update writes into the directions W was *not* using. Intuition first. Picture W as a machine that listens loudly to a handful of input directions and mostly ignores the rest. Its top singular directions are the loud ones. GRPO's update τ could do one of two things: (i) push harder along those same loud directions, merely turning the existing knobs up or down, or (ii) write into the quiet, ignored directions, adding genuinely new behavior. The subspace-projection ratio $\rho_k(\tau)$ measures exactly this: of all the "energy" in τ, what fraction lands inside W's top-k directions? A purely random update would, by chance, land a fraction $(k/r)^2$ of its energy there; that is our dashed null baseline. We find ρ_k sitting *at or below* that chance line at 4B and 8B (Fig. 4b): GRPO is doing (ii), not (i). It edits the under-used subspace and leaves the dominant machinery alone, which is precisely *why* the spectrum (the gains of that dominant machinery) does not move. The effect even *strengthens* with scale: the share of τ lying in W's top-100 directions falls from 0.34% at 3B to 0.04% at 7B, so larger models confine GRPO to an ever-thinner, ever-more- orthogonal sliver of weight space. This is the low-intrinsic-dimensionality picture of fine-tuning (Aghajanyan et al., 2021; Hu et al., 2022), which GRPO exhibits even more sharply than supervised fine-tuning.

The update has its own low-rank structure, and a clean zero control. The spectrum of W is pinned, but τ itself is not spectrally trivial: its *own* singular values decay 2.5°–4.2× across the top ten (4B, averaged over `mlp.down_proj` layers), so each task's update concentrates in a handful of preferred directions rather than spreading isotropically. The leading $\sigma_1(\tau)$ scales with how much a task moves the model: largest for `maze` (0.018) and `knights_knaves` (0.017), smallest for the barely-learnable `mini_sudoku` (0.006). At the bottom end sits a clean control: `cryptarithm` never earns a reward at 4B, so its task vector is *exactly* zero at every layer ($\|\tau\|_F = 0.000$); the geometry pipeline therefore measures learning, not numerical noise. Per-task $\|\tau\|_F$ and $\sigma(\tau)$ values are tabulated in Appendix H.

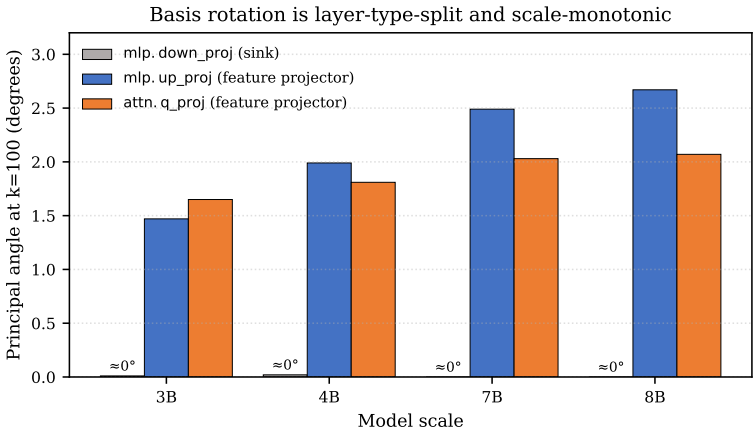
The frozen-sink invariant. The spectrum is pinned, but the singular *vectors*, the directions themselves, can still swivel without changing any σ_i. We quantify that swivel as the principal angle between W_{SFT}'s and W_{GRPO}'s top-100 singular subspaces (0° = identical directions, 90° = fully reoriented). Averaged over all layers it is a mild 1°–3°, but breaking it out by layer type (Table 5, Fig. 4c; the per-task de-aggregation behind these averages is in Appendix H, Table 28) exposes the cleanest invariant in the study: `mlp.down_proj` rotates 0.00°–0.02°, effectively *frozen*, at every scale and in both families, while the feature projectors `mlp.up_proj` and `attn.q_proj` rotate 1.5°–2.7°, growing monotonically with model size. The mechanism is intuitive once named. `down_proj` and `attn.o_proj` are *sinks*: they sum contributions from many neurons back into the residual stream, so their dominant directions are pinned by the aggregate statistics of all inputs and resist reorientation. `up_proj` and `q_proj` are *feature projectors*: they decide which features to compute and what



(a) Spectrum preserved: $\sigma_i(W_{\text{GRPO}}) \approx \sigma_i(W_{\text{SFT}})$.



(b) τ projection onto W's top-k tracks the random baseline $(k/r)^2$.



(c) Basis rotation by layer type and scale: `down_proj` frozen, feature projectors rotate 1.5°–2.7°.

Figure 4: **The universal "how" of GRPO.** The update preserves W's spectrum (a), lives in the directions W was not using (b), and rotates feature-projector bases while leaving the MLP sink `mlp.down_proj` frozen (c). Every panel holds across 3B/4B/7B/8B and both Qwen families.

Table 5: **Basis rotation by layer type** (principal angle at k=100, degrees). `mlp.down_proj` is frozen at every scale; feature projectors rotate, growing with scale.

Layer type	3B	4B	7B	8B
`mlp.down_proj`	0.01°	0.02°	0.00°	0.00°
`mlp.up_proj`	1.47°	1.99°	2.49°	2.67°
`attn.q_proj`	1.65°	1.81°	2.03°	2.07°

to attend to, so they are the natural site for installing task-specific behavior, and they rotate accordingly. The zero-rotation result is specific to the MLP sink: the attention sinks are *not* literally frozen, with `attn.k` rotating 30°–42° and `attn.{v,o}` 50°–70° in the merge analysis of §7.

6.2 Where GRPO edits: same rotation budget, different direction and depth

If every task rotates the feature projectors by the same ∼2°, what makes one task vector differ from another? Two things: the *direction* of the rotation and its *depth*.

Same budget, different direction. The rotation *amount* is almost task-independent: across tasks at a fixed scale it varies by less than 0.1°, as if GRPO spends a fixed "rotation budget" per layer type no matter what it is learning (consistent with the near-uniform $\|\tau\|_F \approx 0.7$–1.2 at 4B reported in §7). What differs is *which way* each task turns. We extract the new directions a task introduces (its *novel subspace*: the part of W_{GRPO}'s top-k basis lying outside W_{SFT}'s; defined in Appendix H). Rotation *direction* is only meaningful relative to another task, so the natural measure is inherently *pairwise*: the principal angle between *two* tasks' novel subspaces (0° = same new direction, 90° = perpendicular), which has no single-task analogue. Figure 5a reports this angle for all six pairs among the four tasks, layer by layer within each layer type, so each cell asks "do tasks A and B rotate this layer the same way?". Tasks genuinely diverge: pairwise angles average 24° on `mlp.down_proj` (its tiny novel subspace is the least task-specific), 39° on `mlp.up_proj`, and 43° on `attn.q_proj`, the most task-adversarial layer type, where individual layers swing from strongly aligned (+0.89 flat cosine) to directly opposing (−0.90) depending on the task pair (Fig. 5a). Those local oppositions are the microscopic seed of the catastrophic cross-task degradation in §4, and, later, of merge damage.

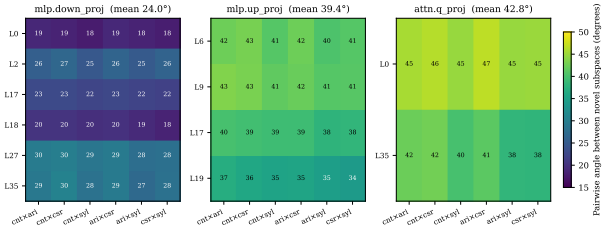
Different depth. Tasks also disagree on *where* along the network's depth to write. A task's depth signature, the profile of $\|\tau\|_F$ across layer indices, is sharply task-specific (Fig. 5b). At 8B, `caesar_cipher` packs 88% of its change into the early layers (fitting a token-level substitution task), the arithmetic tasks (`countdown`, `basic_arithmetic`) edit mid-late layers, and the logic tasks cluster in the mid-early band. These signatures are not fixed: they shift with model scale, and the *direction* of the shift is itself task- and family-dependent (Fig. 5c; the full per-task migration for all nine tasks is in Appendix H, Fig. 8). Comparing the two scales *within* each family, which removes the Qwen2.5-vs-Qwen3 confound, every task we track deepens from 3B to 7B in Qwen2.5, whereas within Qwen3 the movement splits: `basic_arithmetic` and `shortest_path` deepen from 4B to 8B, while `syllogism` and `caesar_cipher` move *shallower* (the latter consolidating 88% of its change into the early layers at 8B). With only two scales per family this fixes the direction of movement, not a monotonic trend; either way, an interference or curriculum analysis calibrated on one model will mispredict another. Finally, depth-profile overlap predicts the sequential-transfer outcomes of §4 *better* than whole-model task-vector cosine, which is < 0.1 for every task pair and explains only ∼10% of the transfer variance (Pearson r=0.32): the conflict is local in depth, and a global average washes it out. The worst transfer in the entire study makes this concrete: `mini_sudoku`→`leg_counting` (−52 at 4B) has near-zero global τ cosine, yet its depth-profile cosine is 0.33 because `mini_sudoku` concentrates its change in `L19`–`21` while `leg_counting` relies on `L12`–`16` and is *negatively* loaded at `L19`–`21`; training on `mini_sudoku` overwrites exactly the layers `leg_counting`'s output stages depend on.

6.3 What this geometry implies for merging

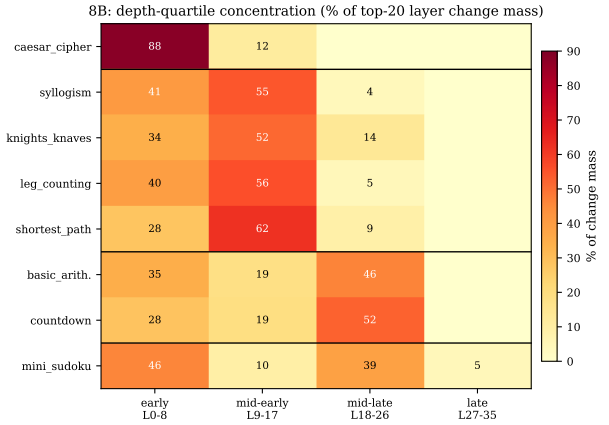
Three of these facts make adding task vectors back onto the base (*task arithmetic*) look theoretically safe. τ is small relative to W (0.06%–0.12% in Frobenius norm), it is near-orthogonal to W's dominant subspace, and different tasks' τ's are near-orthogonal to one another (global cosine 0.00–0.08), so summing them should not produce destructive interference. One fact is even directly actionable: if `mlp.down_proj`'s basis never moves, its τ contribution may be noise that a merge can safely discard. The next section tests both the optimistic geometry and this concrete recipe, and finds that the geometry delivers the recipe, but not the prospective predictability one would hope for.

7 Characterizing Task-Vector Merging

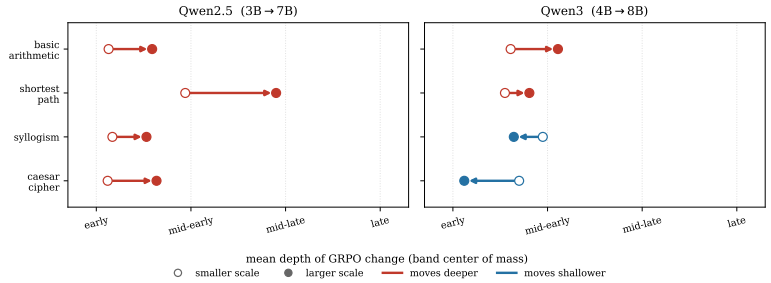
Background. We merge single-task RLVR vectors by task arithmetic (Ilharco et al., 2023), $W_{\text{merged}} = W_{\text{SFT}} + s\sum_i \tau_i$, and ask: do RLVR vectors compose into multi-skill models, and can we predict *which* merges work? We evaluate 19 pairs at 4B (subsets at 3B/8B) and define *pair damage* as the total per-task accuracy a merge loses relative to the two single-task sources, $\text{damage}(A,B) = \max(0, \text{acc}_A^{\text{src}} - \text{acc}^{\text{merge}}) + \max(0, \text{acc}_B^{\text{src}} - \text{acc}^{\text{merge}})$. Damage spans 0.5 to 64.5 points across the 19 pairs. The study ran in six tranches, from a single N-way average to a 19-pair stress test of every candidate predictor. Table 6 lists the damage (and the weight-space sign-agreement we revisit in §7.2) for all 19 pairs; the per-pair layer-selective and full-sweep breakdowns are in Appendix F.



(a) Principal angle between each task *pair*'s novel subspaces (rows = layers within a type, columns = the six task pairs). Larger = the two tasks add more divergent new directions: 24°–43°, largest on `attn.q_proj`.



(b) Depth signatures at 8B: each task concentrates change at a different depth.



(c) Depth migration within each family (mean depth of change, smaller→larger scale). Qwen2.5 tasks (3B→7B) all deepen; in Qwen3 (4B→8B) movement splits: `basic_arithmetic`/`shortest_path` deepen, `syllogism`/`caesar_cipher` move shallower.

Figure 5: **The task-specific "where" of GRPO.** Rotation *magnitude* is task-invariant, but the *direction* of rotation (a) and the *depth* at which change concentrates (b), together with how that depth shifts across scale within each model family (c), are all task-specific.

High-level conclusion. The result splits cleanly into a constructive half and a predictive half. *Construction works*: simply summing two RLVR vectors at full strength produces a usable multi-skill model, and one geometry-derived edit (discard the `mlp.down_proj` contribution) is safe on 40/48 cells we tested. *Prospective prediction fails*: no cheap probe tells you in advance which pairs will merge cleanly. The governing reason for both halves is the same near-orthogonality of RLVR vectors established in §6: because vectors barely interact on average, summing them rarely interferes (so construction is forgiving), but the rare damage that does occur lives at a few localized, per-parameter sign conflicts that every global, averaged metric washes out (so it is unpredictable). The merge-compatibility of two skills is therefore a property of the *interaction* of their two vectors, not an intrinsic property of either task, and it is not predicted by the §4 sequential-transfer matrix ($r \approx 0$): merging and sequential transfer are different operations. The two subsections below give the worked examples and the mechanistic account behind each half.

7.1 What worked, and why

We use `shortest_path+arithmetic` (SP+Arith) as the worked pair for the cross-scale and atomic-versus-direct panels because it is the two-task merge inherited from the §5 curriculum study (there, a plain sum of the two best single-task checkpoints matched every training-based combination we tried), it spans a high-gain spatial task and a frequently-degrading arithmetic one, and confining the expensive layer-selective × 3-scale × atomic/direct sweep to one well-understood pair kept it tractable; the `down_proj` safe-recipe and

Table 6: **Merge damage for all 19 pairs** (4B). *Damage* is the points lost relative to the two single-task sources (lower is better; 0 is lossless). *Sign-agree* is the 3B magnitude-weighted sign-agreement of the two task vectors, the predictor we test in §7.2. Note the wide damage spread at near-identical sign-agreement, e.g. T11 (`sp`+`leg`, 0.5) vs. T15 (`kk`+`sdk`, 64.5), which is what defeats prediction.

Pair	Tasks	sign-agree	damage	Pair	Tasks	sign-agree	damage
T1	sdk+leg	0.530	18.0	T11	sp+leg	0.551	**0.5**
T2	sdk+kk	0.531	22.0	T12	sdk+syl	0.526	19.5
T3	ari+leg	0.564	12.5	T13	ari+sdk	0.547	21.0
T4	ari+syl	0.605	4.5	T14	syl+kk	0.508	20.0
T5	sp+syl	0.536	7.0	T15	kk+sdk	0.531	**64.5**
T6	sp+kk	0.513	18.0	T16	cnt+kk	0.522	10.5
T7	cnt+ari	0.493	24.0	T17	cnt+syl	0.507	40.0
T8	leg+syl	0.538	7.5	T18	cnt+sdk	0.521	45.5
T9	ari+kk	0.512	22.0	T19	sp+sdk	0.530	51.0
T10	cnt+leg	0.500	**0.5**				

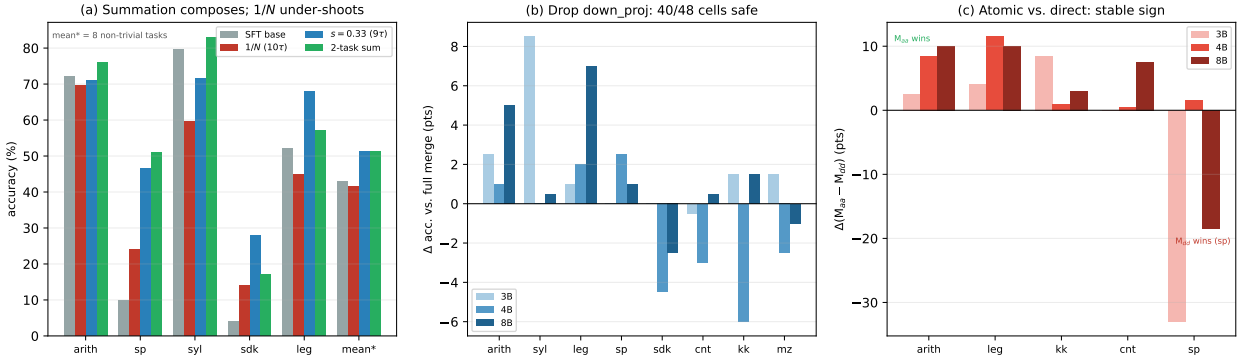


Figure 6: **The three constructive results.** (a) Summation composes: a 2-task sum (s=1.0, green) holds both skills and beats $1/N$ averaging (red) on 9/10 tasks, while $1/N$ under-shoots every task; the binding constraint is the total displacement budget, not the source count. (b) Dropping the `mlp.down_proj` contribution is safe: for SP+Arith the change versus the full merge is neutral-to-positive at 3B/4B/8B (bars shown; at or above 0 is lossless), and across the 4B pair set it is lossless on 40/48 (pair, task) cells, the one universally safe recipe. (c) Atomic (M_{aa}) versus direct (M_{dd}) vectors compose with a per-task sign that is stable across scale: M_{aa} wins on arithmetic/leg-counting/knights-knaves/countdown at every scale, M_{dd} wins on `shortest_path`. Data transcribed from Tables 7 and 8.

the null-predictor claims below are then validated across all 19 pairs, with the per-pair budget sweep and layer-selectivity breakdowns of the other combinations tabulated in Appendix F.

Simple summation composes skills (near-orthogonality). Summing two single-task vectors at s=1.0 yields a model that holds both skills: SP+Arith lifts `shortest_path` from 10% (SFT base) to 51% while keeping arithmetic at 76% (Table 7, Figure 6a). The mechanism is the §6 finding that RLVR τ's are near-orthogonal both to W and to each other: two skills written into nearly disjoint directions add without overwriting one another. This is not special to SP+Arith: we summed all 19 pairs (Table 6), and composition is the default across them, with median damage 19.5 points, 15/19 pairs below 25, and two near-lossless pairs (`cnt`+`leg` and `sp`+`leg`, both 0.5). Catastrophic clashes are the exception, confined to four `mini_sudoku`- and `countdown`-heavy outliers (`kk`+`sdk` 64.5, `sp`+`sdk` 51, `cnt`+`sdk` 45.5, `cnt`+`syl` 40). Composition is thus the *default* outcome of summation, and catastrophic clashes are the exception, not the rule.

Scale beats N; $1/N$ averaging is a dead end. The TIES/DARE-canonical recipe of averaging all ten τ's at s=$1/N$=0.1 under-shoots: it loses to a 2-task sum (s=1.0) on 9/10 tasks (mean of 8 non-trivial tasks: 41.6% vs. 51.2%; Table 7, Figure 6a). The theory follows from the same near-orthogonality: because

Table 7: **Merge budget, not source count, matters** (4B, accuracy). $1/N$ averaging under-shoots; 2-task and 9-task (s=0.33) merges tie at ≈ 3 τ-equivalents.

Task	SFT	$1/N$ (10τ)	s=0.33 (9τ)	2-task (SP+Arith)
basic_arithmetic	72	69.5	71.0	**76.0**
shortest_path	10	24.0	46.5	**51.0**
syllogism	79.5	59.5	71.5	**83.0**
mini_sudoku	4	14.0	**28.0**	17.0
leg_counting	52	45.0	**68.0**	57.0
Mean (8 non-trivial)	42.9	41.6	**51.3**	**51.2**

Table 8: **Atomic vs. direct composition** $\Delta(\mathrm{M}_{aa} - \mathrm{M}_{dd})$ (pts). Per-task sign is cross-scale stable; M_{dd} wins only on `shortest_path`.

Task	3B	4B	8B	Pattern
basic_arithmetic	+2.5	+8.5	+10.0	M_{aa} wins at every scale
leg_counting	+4.0	+11.5	+10.0	M_{aa} wins at every scale
knights_knaves	+8.5	+1.0	+3.0	M_{aa} wins at every scale
countdown	0.0	+0.5	+7.5	M_{aa} wins
shortest_path	**−33.0**	+1.5	**−18.5**	**M_{dd} wins**

vectors do not destructively interfere, the binding constraint is the *total displacement budget* ($s \cdot N \approx 2$–3 τ-equivalents), not the number of sources. Dividing by N does not avoid interference (there is little to avoid); it merely shrinks every skill below the displacement it needs, so all skills under-shoot at once.

Always drop `mlp.down_proj` (the actionable recipe). Dropping `down_proj` contributions from a merge is neutral-to-positive on 40 of the 48 (pair, task) cells we evaluated at 4B (Figure 6b). Each *cell* is one task's accuracy under the `down_proj`-dropped merge compared against the full merge for one pair, and *neutral-to-positive* means the dropped merge stays within 3 points of, or exceeds, the full merge, i.e. the drop is lossless; the 48 cells cover the 4B merge pairs across the eight non-trivial tasks, and the recipe is additionally mean-positive at 3B and 8B. This is the one universally safe, geometrically motivated merge recipe we found, and it survives every scale and every pair we tested. The theory is a direct operationalization of the §6 frozen-basis invariant: the `mlp.down_proj` basis rotates 0° under RLVR, so each task vector's `down_proj` component points along directions the model never repurposes while learning the skill. That component therefore carries no skill-specific signal, only summed numerical noise that can fight across vectors, so discarding it removes a source of conflict at no capability cost. More aggressive masking (`up_proj`+`gate_proj` only) works for some pairs and fails for others (the per-pair layer-selective breakdown across six further pairs is in Appendix F), and TIES and DARE (Yadav et al., 2023; Yu et al., 2024) do not consistently beat vanilla summation (within ±2 pts on three tested pairs): once `down_proj` is gone, the remaining attention circuit is genuinely load-bearing for some pairs and cannot be pruned generically.

Atomic vs. direct vectors compose differently (cross-scale stable). Merging two atomic-curriculum vectors (M_{aa}) versus two direct-GRPO vectors (M_{dd}) for the same task pair gives a per-task sign that is stable across scale: M_{aa} wins on feature-routing tasks (arithmetic, leg_counting, knights_knaves, countdown) at every scale, while M_{dd} wins on `shortest_path` at every scale (Table 8, Figure 6c). The mean magnitude is scale-specific (+7.9 at 4B, +0.3 at 8B, −3.3 at 3B), but the sign pattern is robust. Our hypothesis for the split mirrors the §5 atom decomposition: the atomic curriculum front-loads grounding (L1) and writes the skill into the same feature-routing MLP directions that compose additively, whereas `shortest_path`'s gain is carried by attention-circuit restructuring that direct GRPO captures more faithfully, so its atomic vector composes worse. Pairing this with the §5 regime rule yields a heterogeneous training-then-merging recipe: train each skill in its ARS-recommended regime, then sum.

Table 9: **Predictor decay.** Entries are Pearson correlations r between each candidate predictor and merge damage. A *useful* predictor needs large magnitude $|r|$ (near ± 1); $r \approx 0$ means no predictive power, so for a predictor larger $|r|$ is "better". The sign only encodes direction (negative r: a higher predictor value implies lower damage). Both predictors that look strong at n=7 decay toward 0 by n=19, i.e. lose their power; the right column lists predictors null at every n.

Predictor (Pearson r)	n=7	n=12	n=19	Null at every n
3B sign-agreement → damage	−0.81	−0.45	−0.27	Phase-1 transfer ($r \approx 0$)
3B L1-ARS → damage	−0.76	+0.06	n/a	global τ cosine
routing probe → group	n/a	−0.31	n/a	per-layer flat-cosine (+0.23)
best single residual feature	n/a	$< \|0.5\|$	n/a	τ magnitude / eff. rank

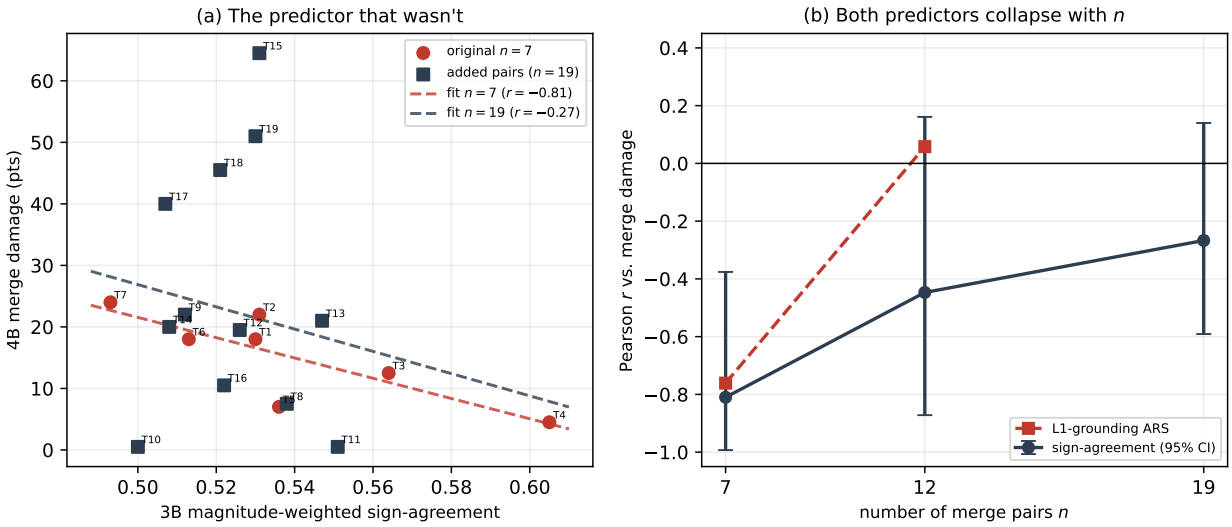


Figure 7: **No prospective predictor of merge damage survives.** (a) 3B sign-agreement vs. 4B merge damage: the steep fit on the original n=7 pairs (r=−0.81) flattens to r=−0.27 over all n=19; outliers like T10/T11 (damage 0.5) and T15 (damage 64.5) sit at ordinary sign-agreement. (b) Both candidate predictors decay toward zero as pairs are added: sign-agreement −0.81→−0.27 (95% CI crossing zero for every $n \geq 12$) and L1-grounding ARS flips sign (−0.76→+0.06).

7.2 What did not work, and where it broke

No cheap predictor of which pairs merge cleanly (broke at larger n). The natural goal, a pre-probe that flags clashing pairs before any merge, proved unreachable. An earlier draft of this work reported two predictors of 4B merge damage estimated on the original n=7 pairs: a weight-space *magnitude-weighted sign-agreement* at 3B (r=−0.81) and a behavioral *L1-grounding ARS* at 3B (r=−0.76). We ran 12 more pairs (n=19) specifically to stress-test them *before* submission, and that is exactly where they broke: both collapse monotonically with sample size (Table 9, Figure 7). Sign-agreement decays −0.81→−0.45→−0.27 (bootstrap CI crosses zero at every $n \geq 12$), and L1-ARS *flips sign* at n=12 (−0.76→+0.06). Our hypothesis for why they ever looked strong is a favorable-small-sample artifact: with only seven pairs, a single monotone ordering is easy to hit by chance, and the steep n=7 fit is dominated by a few points; the population correlation is a weak directional ≈ -0.3 at best, too noisy to calibrate a go/no-go decision.

Task-intrinsic merge "groups" do not exist (broke when partners changed). An earlier draft classified *tasks* as "MLP-only-sufficient" (G1) or "needs-full-attention" (G2) for merging, a clean task-level taxonomy. Re-running layer-selective merges on 6 more pairs broke it: `shortest_path` is G2 with `arithmetic` but G1 with `knights_knaves`; `syllogism` flips the other way; only `mini_sudoku` is stably G2. The hypothesis that fits this is that compatibility is *interactional*: which layers a merge needs is a property of how two

specific vectors overlap at specific parameters, not a label you can attach to either task in isolation, so it cannot be read off task features. The corroborating nulls follow the same logic: a task-level routing probe predicts group at only $r=-0.31$, and a residual search over ten candidate features finds no single $|r| > 0.5$ and no two-feature $R^2 > 0.55$ at $n=12$ (a "task-incompatibility" feature reaches $r=0.82$ on the full sample but $r=0.06$ under leave-one-out, entirely overfit on task identity). Completing the §6 picture, `attn.{k,v,o}` are *not* frozen (k rotates 30°–42°, v/o 50°–70°), which is why some pairs genuinely need the full attention circuit and no fixed mask generalizes.

Why no global metric can see the damage (the unifying hypothesis). The same near-orthogonality that makes summation work is what makes damage invisible. §6 established that RLVR τ's are nearly orthogonal both to W and to each other, so any *averaged* statistic of two vectors looks essentially identical whether or not they will clash. Damage, when it occurs, lives at a *localized, per-parameter* level: sign conflicts at a handful of high-magnitude positions that a global average dilutes to nothing. This predicts exactly what we see: two tasks with perfectly normal sign-agreement can still merge catastrophically (`knights_knaves`+`mini_sudoku`: damage 64.5, KK drops 47 pts). It also predicts that a useful predictor, if one exists, would have to be a *localized per-parameter* conflict measure rather than any of the global summaries we tried. The honest conclusion: RLVR 2-vector merging has a *characterized but not prospectively predictable* structure. We regard this as a contribution. The merging literature has implicitly assumed task-intrinsic compatibility and cheap pre-probes, and our $n=19$ evidence rules out both for reasoning RLVR vectors.

8 Limitations

(1) **Family confound:** 3B/7B are Qwen2.5, 4B/8B are Qwen3, so some "scale" effects may be family effects; non-Qwen replication (Llama-3, Gemma-2) is future work. (2) **Sample size:** merge analysis is $n=19$ pairs at one focal scale (4B); the null results are robust to this n but a still-larger sweep could surface a weak predictor we missed. (3) **Seed variance:** training-seed variance is spot-checked, not exhaustively characterized. (4) **Atom design:** ARS is only as good as the hand-authored atoms, as `mini_sudoku` shows. (5) **Causality:** Phase-3 geometry is correlational; we do not run interventional experiments (e.g. projecting out shared directions during multi-task training).

9 Conclusion

GRPO on small reasoning models is not free improvement. It degrades a third of task pairs, but its weight-space action has a remarkably clean, scale-invariant signature, and a cheap probe (ARS) predicts when and how to apply it. That geometry yields a robust merge recipe (drop `mlp.down_proj`) and a cross-scale composition effect (atomic vs. direct), but *not* a prospective merge-damage predictor: two predictors we initially believed at $n=7$ collapsed at $n=19$. The result is an empirical characterization of RLVR training dynamics that is honest about what it can and cannot predict, and we hope a useful map for practitioners and a caution to the model-merging literature.

Five universal empirical laws. Across all phases and scales: (1) GRPO is spectrum-preserving; (2) τ is near-orthogonal to W's top subspace; (3) the `mlp.down_proj` basis is always frozen; (4) `down_proj` τ contributions are always droppable from a merge; (5) single-task GRPO produces a τ of Frobenius magnitude $\sim$0.7–1.2 at 4B, insensitive to task identity, with only the direction and depth varying.

The connective thread. ARS diagnoses *what RLVR needs* (§5); the weight geometry shows *what GRPO structurally does* (§6); and that geometry makes merging tractable (drop `down_proj`) but only *partly* predictable (§7). The bridge we once hoped for, a grounding bottleneck unifying single-task success with merge compatibility, did not survive (§7.2): ARS is a valid *single-task training-regime* predictor, but not a *merge-compatibility* predictor. We think honesty about this boundary is more useful to the field than an over-claimed unification.

Practical recipe. (i) Run the ARS probe (∼1 hr); pick the regime by band, watching for reasoning compression on long-chain tasks. (ii) Train single-task τ's, using atomic curricula for feature-routing tasks and direct GRPO for spatial-search tasks. (iii) Merge with the `all_but_down` mask at $s{\approx}1.0$ for two vectors (scale down for more); *measure* each merge. No cheap probe substitutes.

Reproducibility. All SFT/GRPO checkpoints, ARS probes, merge checkpoints, evaluation outputs, and analysis scripts are released. Appendices give full per-scale transfer matrices (A), per-task atom breakdowns (B), the shortest-path deep dive (D), the MMLU forgetting study (E), the full 19-pair merge table (F), and hyperparameters (G).

References

Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning (ICML)*, 2009.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

Michael Matena and Colin Raffel. Merging models with fisher-weighted averaging. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Zeju Qiu, Weiyang Liu, Haiwen Feng, Yuxuan Xue, Yao Feng, Zhen Liu, Dan Zhang, Adrian Weller, and Bernhard Schölkopf. Controlling text-to-image diffusion by orthogonal finetuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Reasoning Gym Contributors. Reasoning gym: Procedural reasoning environments with verifiable rewards. `https://github.com/open-thought/reasoning-gym`, 2025.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. Trl: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, 2022.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *International Conference on Machine Learning (ICML)*, 2024.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations (ICLR)*, 2023.

A 7B/8B Transfer Matrices and the Learning-Rate Sweep

The four matrices below complete the cross-scale picture (`caesar_cipher` and `cryptarithm` omitted as before). At the aggressive lr=5×10^{-6} (v1) the 7B/8B models are the most volatile in the study, both the largest gains and the largest harms; the gentle lr=1×10^{-6}/16-generation setting (v2) flattens the matrix to within ±8 pts everywhere, eliminating catastrophic harm but also nearly all learning.

Table 10: **Qwen2.5-7B v1** (lr=5×10^{-6}, 8 generations).

Trained on	cnt	mz	sdk	pth	ari	syl	kk	leg
countdown	+60*	−4	−6	0	−2	+6	+8	−8
maze	−4	+10*	−6	−4	−2	+4	0	−10
mini_sudoku	+2	−4	+16*	−4	−6	−8	+4	−6
shortest_path	−4	−2	−4	+36*	−8	−24	0	−2
basic_arith	−2	+2	−12	−4	−30*	−2	0	−30
syllogism	0	+6	−12	−4	−46	+26*	−28	−32
knights_knaves	−2	−2	−10	−4	−32	−8	+28*	−32
leg_counting	−4	+2	−4	−4	−10	−2	+2	+8*
Baseline	4%	14%	12%	4%	60%	64%	32%	32%

The aggregate learning-rate effect is summarized below.

B Per-Task Atom Breakdowns and the ARS Recipe

The ARS measurement recipe. ARS is a pure inference probe (no gradients, no training data). For each task: (1) hand-author 6–14 atom templates split between L1 (grounding) and L2 (local reasoning), each a deterministic function of a `reasoning-gym` sample with a programmatically verifiable answer; (2) draw 50 valid (question, ground-truth) pairs per atom (seed 9999, oversampling for atoms that reject ill-fitting samples); (3) zero-shot generate from the SFT base with a fixed "answer concisely inside `<answer>` tags" prompt, greedy decoding, 128 max new tokens, and extract the *last* `<answer>` match (first-match was fooled by echoed template tags); (4) score each atom with a *lenient*, type-aware verifier (numeric/list/character/token canonicalization, not strict string equality); (5) average into L1_avg, L2_avg, and ARS = mean(L1 ∪ L2). Cost: ≈ 8–10 min/task on one GPU; the 7-task probe runs in ≈ 1 hour.

Table 11: **Qwen3-8B v1** (lr=5×10^{-6}, 8 generations). `mini_sudoku` reaches +60 (0%→60%); `maze` is a corrupted zero-reward run.

Trained on	cnt	mz	sdk	pth	ari	syl	kk	leg
countdown	+50*	−10	+22	+8	−4	−2	+2	−4
maze	−24	−10*	0	−8	−8	−22	−30	−40
mini_sudoku	+18	+10	+60*	+16	+10	+6	+6	−4
shortest_path	−4	+10	+8	+36*	+2	−8	+6	0
basic_arith	−14	+4	+14	+4	−2*	−4	−4	−8
syllogism	−16	−6	+6	−2	−2	+18*	−6	−16
knights_knaves	−18	−8	+16	+4	−2	0	−6*	−10
leg_counting	+6	−2	0	+10	+4	−10	0	0*
Baseline	32%	10%	0%	8%	64%	78%	64%	48%

Table 12: **Qwen2.5-7B v2** (lr=1×10^{-6}, 16 generations).

Trained on	cnt	mz	sdk	pth	ari	syl	kk	leg
countdown	−2*	+4	+4	0	0	+2	+4	−6
maze	−4	+6*	+6	+4	0	+2	+2	−6
mini_sudoku	−4	0	+8*	+4	0	−2	+2	−2
shortest_path	−2	+2	+8	+8*	−2	0	+2	−6
basic_arith	−2	+2	+2	+4	−4*	+2	0	−2
syllogism	−4	+2	+4	0	−2	+8*	+2	−4
knights_knaves	−2	0	+6	+2	0	−4	+4*	0
leg_counting	−4	+2	+4	+4	0	0	−4	+2*
Baseline	4%	12%	4%	2%	60%	64%	30%	36%

Table 13: **Qwen3-8B v2** (lr=1×10^{-6}, 16 generations). Note the systematically negative `kk` column: GRPO on anything degrades this fragile 74% task.

Trained on	cnt	mz	sdk	pth	ari	syl	kk	leg
countdown	+4*	−2	+6	−8	+2	0	−12	−2
maze	0	0*	+4	−6	+2	0	−10	−2
mini_sudoku	+4	+2	+2*	−2	+4	+4	−6	0
shortest_path	−4	−4	0	−4*	+2	+4	−8	−2
basic_arith	+2	+2	0	−6	+4*	+2	−8	−2
syllogism	+6	−2	0	−2	+4	+2*	−10	0
knights_knaves	+6	−4	+4	−6	+4	0	−14*	−2
leg_counting	+2	−4	0	−6	+4	+2	−18	0*
Baseline	22%	10%	0%	12%	62%	74%	74%	52%

Table 14: **The learning-rate effect** (v1 = $5e{-}6$, v2 = $1e{-}6$/16 gen).

Metric	7B v1	7B v2	8B v1	8B v2
Max diagonal gain	+60	+8	+60	+4
Worst diagonal harm	−30	−4	−10	−4
Mean $\lvert\Delta\rvert$	8.9%	3.0%	11.2%	3.8%
Cells with $>10\%$ harm	14	0	11	2

Table 15: **Per-task ARS summary** (Qwen3-4B SFT base, 50 samples/atom). L1 = grounding floor, L2 = local-reasoning floor.

Task	#L1	#L2	L1_avg	L2_avg	ARS	Reading
syllogism	4	3	96%	52%	74%	aces grounding; single-hop validity is the gap
knights_knaves	4	4	57%	68%	62.5%	unusual L2>L1; cannot attribute statements
basic_arithmetic	4	4	67.5%	65%	66%	both floors clear; fails via reasoning compression
countdown	3	3	34%	78%	56%	weak L1, strong L2; bottleneck is L3 search
maze	4	3	14%	27%	20.7%	cannot locate start/goal
shortest_path	4	3	0%	27%	14%	cannot read the grid at all
mini_sudoku	4	4	24%	15.5%	19.8%	atoms miss the constraint-propagation bottleneck

Table 16: Atom-level base accuracy, high/medium-ARS tasks.

Task	Atom	Layer	Base
syllogism	premise1_subject	L1	100%
	premise2_subject	L1	100%
	quantifier_type	L1	86%
	conclusion_predicate	L1	98%
	single_hop_inference	L2	46%
	negation	L2	42%
	validity_check	L2	68%
knights_knaves	count_inhabitants	L1	66%
	list_names	L1	90%
	speaker_identification	L1	0%
	terminology	L1	72%
	statement_about	L2	72%
	if_sage_then	L2	96%
	if_fool_then	L2	100%
	consistency_check	L2	4%
basic_arithmetic	number_extraction	L1	62%
	operator_identification	L1	70%
	term_count	L1	40%
	has_parentheses	L1	98%
	single_op_eval	L2	76%
	order_of_operations	L2	2%
	innermost_eval	L2	82%
	sign_handling	L2	100%
countdown	number_inventory	L1	10%
	target_reading	L1	90%
	how_many_numbers	L1	2%
	single_op	L2	94%
	two_number_reachability	L2	50%
	distance_to_target	L2	90%

Table 17: Atom-level base accuracy, low-ARS tasks. Note the floor-zero grounding for the grid tasks.

Task	Atom	Layer	Base
maze	locate_start	L1	0%
	locate_goal	L1	0%
	read_cell	L1	54%
	grid_size	L1	2%
	valid_moves	L2	8%
	is_move_valid	L2	62%
	shortest_path_length	L2	12%
shortest_path	locate_start	L1	0%
	locate_goal	L1	0%
	read_cell	L1	0%
	grid_dimensions	L1	0%
	valid_neighbors	L2	0%
	is_move_legal	L2	54%
	direction_to_goal	L2	28%
mini_sudoku	read_cell	L1	24%
	count_empty	L1	0%
	row_values	L1	30%
	col_values	L1	42%
	missing_in_row	L2	16%
	missing_in_col	L2	22%
	can_place	L2	20%
	only_option	L2	4%

C Per-Task Atomic Curriculum Experiments

Section 5 reports the headline result from one summary table. Here we give the full four-condition experiment behind every task, paired with the per-atom bottleneck (the atom tables in Appendix B) that explains it. All conditions share the GRPO recipe of Appendix G (lr=5×10^{-6} constant, DAPO loss, β=0, KL=0, 8 generations/prompt, temperature 1.0) and matched compute:

- **Base** — the SFT checkpoint, no GRPO.
- **Direct-RLVR** — 300 GRPO steps on the full task.
- **Atomic-Only** — 100 steps on L1 atoms → 100 steps on L2 atoms, then *zero-shot* evaluation on the full task (no full-task training at all).
- **Atomic-Warm** — L1 (100) → L2 (100) → full task (100), 300 steps total.

Table 18: **Full atomic-curriculum results, all 7 tasks.** Full-task eval accuracy. Bold = best regime per task. Δ is best minus Direct-RLVR, i.e. how much the atomic curriculum buys over training the full task directly.

Task	ARS	L1	L2	Base	Direct	At.-Only	At.-Warm	Best	Δ
syllogism	74%	96%	52%	79.5	96.5	81.0	**97.0**	At.-Warm	+0.5
knights_knaves	62.5%	57%	68%	78.0	**97.5**	79.0	95.0	Direct	—
basic_arithmetic	66%	67.5%	65%	72.0	32.0	**78.0**	70.0	At.-Only	**+46**
countdown	56%	34%	78%	26.0	**∼70**	24.0	26.0	Direct	—
maze	20.7%	14%	27%	21.5	1.0	**41.0**	38.5	At.-Only	**+40**
shortest_path	14%	0%	27%	10.0	44.0	12.0	**46.0**	At.-Warm	+2
mini_sudoku	19.8%	24%	15.5%	4.0	**25.5**	17.0	24.0	Direct	—

High-ARS tasks (> 60%). *syllogism* (ARS 74%) aces grounding (L1 96%) but is mediocre at single-hop validity (L2 52%); both Direct-RLVR and Atomic-Warm reach ≈ 97%, so GRPO supplies the missing L2/L3 inference and atoms add only +0.5. *knights_knaves* (ARS 62.5%) shows the unusual L2>L1 pattern: the model knows the if-sage/if-fool rules (96%/100%) but cannot attribute statements to speakers (`speaker_identification` 0%, `consistency_check` 4%); despite these gaps the 78% base plus Direct-RLVR (+19.5) learns the attribution implicitly, and Atomic-Only barely moves it. *basic_arithmetic* (ARS 66%) is the instructive exception in this band: both floors clear, yet Direct-RLVR *destroys* it (72 → 32). The bottleneck is not grounding but `order_of_operations` (2%), and full-task GRPO triggers reasoning compression (§5); Atomic-Only is the only regime that fixes the gap without compressing, reaching 78% (+46 over Direct).

Medium-ARS task (40–60%). *countdown* (ARS 56%) has weak L1 (cannot inventory or count the numbers, `number_inventory` 10%, `how_many_numbers` 2%) but strong L2 arithmetic (`single_op` 94%). The true bottleneck is L3: the multi-step search for a combination reaching the target, which atoms do not teach. Only Direct-RLVR moves the needle (26 →∼ 70); both atomic regimes stay at base. This is the band where the decision rule says "skip the atoms."

Low-ARS tasks (< 25%). *maze* (ARS 20.7%) is the headline rescue: grounding is at the floor (`locate_start`/`locate_goal` 0%), so Direct-RLVR collapses to 1% — *below* the 21.5% base, because GRPO with no grounding has no correct trajectory to reinforce and thrashes. Atomic-Only teaches the grounding directly and reaches 41% (+40), with no full-task training at all. *shortest_path* (ARS 14%, the lowest) has complete L1 failure (every grounding atom at 0%); short Atomic-Only atoms fail (12%) because they teach grounding but compress reasoning, whereas Atomic-Warm (46%) and the multi-step variants of Appendix D (55.5%) succeed. *mini_sudoku* (ARS 19.8%) is the one task that breaks the rule: despite low ARS, Direct-RLVR (25.5%) beats both atomic regimes. The cause is diagnostic rather than a counterexample to the mechanism: sudoku's real bottleneck is multi-cell constraint propagation, a holistic L2–L3 operation, and our single-hop cell-reading atoms simply do not probe it, so the ARS we measure is uncorrelated with

the actual difficulty. The lesson is the method's honest boundary: *ARS is only as predictive as the atoms are well chosen.*

D The Shortest-Path Deep Dive (15+ Conditions)

We took `shortest_path` (the lowest-ARS task, 14%) from 10% base to 55.5% through 16 training conditions (Table 19). Four reusable principles emerged.

Table 19: **Shortest_path: all 16 conditions** (Qwen3-4B). RFT-Base is best.

Condition	Acc.	Note
Base (SFT)	10%	reference
Direct-RLVR	44%	Phase-1 baseline (300 steps)
Atomic-Warm-300 (short L2 atoms)	46%	short atoms + full GRPO
A: progressive full-task (3×3→4×4→full)	46.5%	no atoms
B: L2-long atoms → full	51.5%	path-tracing atoms, hard transition
B2: B with 300+300 steps	47%	more steps hurt
C: partial-credit reward	35.5%	reward too easy for GRPO
D: progressive + partial credit	24%	worst; PC kills gradient signal
E: L2-long atoms → progressive full	27.5%	progressive after atoms hurts
F: feasibility atoms → full	42%	fewer false-infeasible, more wrong paths
G: heavy GRPO+SFT interleave	5.5%	SFT collapsed the model
G2: light GRPO+SFT interleave	40%	improving (500 samples, lr $1e{-}6$)
G3: G2 + 150 GRPO	45%	matches Direct-RLVR
L: feasible-only training	40.5%	lost infeasibility detection
RFT on B's checkpoint	51.5%	on-policy SFT adds nothing to a trained model
RFT-Base	**55.5%**	B's traces → base SFT → GRPO 300

(i) Long-answer atoms beat short. Short yes/no atoms induce reasoning compression (Atomic-Only 12%, below base); multi-step path-tracing atoms reach 51.5% (condition B). **(ii) One hard difficulty transition** (atoms → full grids) beats any progressive easing (E 27.5%, which over-fits easy instances). **(iii) Partial-credit rewards hurt GRPO** (C 35.5%, D 24%) by removing the advantage variance the group-relative estimator needs. **(iv) RFT-Base** (generate verified-correct traces from a trained model, SFT the *base* on them, then GRPO) is best at 55.5% because it spends the full step budget on the real task from a clean initialization.

Error analysis of condition B (51.5%). On 100 eval problems the failures were 23 false-infeasible (claims no path when one exists), 25 wrong-path, and 6 missed-infeasible; feasible-problem accuracy was only 25% vs. 83% on infeasible ones. Eliminating the false-infeasible "give-up reflex" alone would lift accuracy to 68.3%, motivating the feasibility-atom and feasible-only experiments (F, L), which reduced false-infeasibles but traded them for wrong paths.

E MMLU: No Catastrophic Forgetting

To check whether reasoning RLVR erodes general knowledge, we evaluated 48 checkpoints, spanning 3 sizes, all 4 phases (SFT → single-task GRPO → atomic curriculum → task-vector merging), and 10 tasks, on three math MMLU subsets (Hendrycks et al., 2021): `abstract_algebra` (100 q), `elementary_mathematics` (378 q), and `high_school_mathematics` (270 q). **No condition produces catastrophic forgetting:** every checkpoint stays within ±3% of its SFT baseline (Tables 20–21).

Table 20: SFT baselines and per-size GRPO ranges on math MMLU (`abstract_algebra` / `elementary` / `high_school`).

Size	SFT baseline	GRPO range (all tasks)	Verdict
3B	47.0 / 60.8 / 50.4	44–47 / 59–61 / 50–51	slight (1–3%) on abstract algebra
4B	56.0 / 70.6 / 51.5	56–58 / 70–71 / 51–53	maintained or improved
8B	52.0 / 70.4 / 51.9	53–55 / 70–71 / 50–52	maintained

Table 21: Phase-1 single-task GRPO (4B) on math MMLU, all 10 tasks maintain the SFT-4B baseline (56.0 / 70.6 / 51.5).

GRPO task (4B)	abstract_algebra	elementary	high_school
basic_arithmetic	57.0	70.9	52.2
countdown	56.0	70.9	52.2
shortest_path	58.0	69.8	51.9
syllogism	58.0	70.4	51.1
maze	57.0	69.8	51.9
knights_knaves	57.0	70.4	52.2
mini_sudoku	56.0	70.6	52.2
leg_counting	58.0	70.6	52.6
caesar_cipher	56.0	70.4	51.5
cryptarithm	56.0	70.6	51.5

The atomic-curriculum (Phase 2) and layer-selective-merge (Phase 4) checkpoints behave identically (all within ±3%; omitted for space). GRPO-based reasoning training on these tasks is thus safe with respect to general math knowledge.

F Full Merge Sweep and Layer-Selectivity Tables

The per-pair damage and sign-agreement for all 19 pairs is given in the main text (Table 6). Here we give the full Part-1 budget sweep and the per-pair layer-selectivity breakdown that underlies the pair-dependence finding (§7.2).

Table 22: **Full Part-1 merge sweep** (4B, 10-task accuracy): $1/N$ averaging vs. the s=0.33 9-vector merge vs. the 2-vector SP+Arith sum.

Task	SFT	$1/N$ (10τ, s=0.1)	s=0.33 (9τ)	2-task (SP+Arith)
basic_arithmetic	72	69.5	71.0	**76.0**
countdown	26	39.0	39.0	**39.5**
shortest_path	10	24.0	46.5	**51.0**
maze	21.5	7.5	4.0	**8.5**
mini_sudoku	4	14.0	**28.0**	17.0
syllogism	79.5	59.5	71.5	**83.0**
knights_knaves	78	74.0	**82.0**	77.5
leg_counting	52	45.0	**68.0**	57.0
caesar_cipher	0	0.0	0.0	0.5
cryptarithm	0	0.0	0.0	0.0
Mean (8 non-trivial)	42.9	41.6	**51.3**	**51.2**

Table 23: **Layer-selectivity is pair-dependent** (F34). Per-(pair, task) Δ vs. the full merge for the `up`, `up+q`, and `all_but_down` masks, and the resulting Group label. `shortest_path` and `syllogism` flip group with their partner; only `mini_sudoku` is stably G2.

Pair	Task	Δup	Δup+q	Δabd	Group
T1 sdk+leg	mini_sudoku	−8.0	−9.0	+3.5	G2
T1 sdk+leg	leg_counting	0.0	+1.0	−8.0	G1
T2 sdk+kk	mini_sudoku	−24.0	−27.0	−16.5	G2
T2 sdk+kk	knights_knaves	+9.0	+7.0	−11.5	G1
T3 ari+leg	basic_arithmetic	+23.0	+23.0	+11.5	G1
T3 ari+leg	leg_counting	+2.0	−4.5	+6.5	G1
T4 ari+syl	basic_arithmetic	+16.5	+16.0	+15.0	G1
T4 ari+syl	syllogism	−6.0	−5.5	+0.5	G2
T6 sp+kk	shortest_path	+23.0	+22.5	+31.0	**G1**
T6 sp+kk	knights_knaves	0.0	−1.0	+1.0	G1
T7 cnt+ari	countdown	+27.0	+27.0	+32.0	G1
T7 cnt+ari	basic_arithmetic	+10.5	+11.0	+10.0	G1

Per-task mean damage is descriptive, not predictive. Averaging each task's merge damage over the pairs it appears in (Table 24) shows that some tasks are, on average, riskier merge partners: `knights_knaves` (20.7) and `mini_sudoku` (19.8) sit far above `leg_counting` (7.8) and `shortest_path` (8.5). This is *not* a usable predictor: the per-task ranges are wide and overlapping (e.g. `knights_knaves` spans 18–64.5, the 64.5 being the `kk`+`mini_sudoku` catastrophe), and the pair-dependence result above (F34) shows a task's contribution flips with its partner. The table is a summary of where damage tended to land, consistent with the headline that merge compatibility is a property of the *pair*, not the task.

Table 24: **Per-task mean merge damage** (4B, over the n=12 pairs in the residual study). Reported for description only; the wide min–max ranges and the pair-dependent layer split (previous table) make this non-predictive for any individual pair.

Task	# pairs	Mean	Min	Max
leg_counting	5	7.8	0.5	18.0
shortest_path	3	8.5	0.5	18.0
syllogism	4	9.6	4.5	19.5
countdown	2	12.2	0.5	24.0
basic_arithmetic	4	15.8	4.5	24.0
mini_sudoku	3	19.8	18.0	22.0
knights_knaves	3	20.7	18.0	22.0

G Hyperparameters

Table 25: GRPO, SFT, and evaluation hyperparameters (shared across phases unless noted).

GRPO (DAPO)		
learning rate	5×10^{-6} constant	(1×10^{-6} in 7B/8B v2)
max steps	300 single-task	100 per atomic stage
num generations	8	16 in v2
per-device batch / grad-accum	1 / 1	
β / KL coeff	0 / 0	pure GRPO, no reference model
ϵ_{high}	5.0	asymmetric clip
temperature	1.0	
max completion length	2048 full	256 atoms
max prompt length	512	1024 for shortest_path
mask truncated completions	true	
stop strings	`</answer>`	
seed	42 (train)	9999 (eval)
SFT warm-up		
learning rate / epochs	2×10^{-5} / 3	
batch / grad-accum	2–4 / 2–4	
max seq length / data	1024 / 1000 samples	task-agnostic format warm-up
Evaluation		
samples / seed	200 / 9999	50 in Phase 1
max new tokens	1024	2048 in Phase 1
DDP	8 GPUs, all_reduce	padding side left

Merging baselines. TIES (Yadav et al., 2023) prunes low-magnitude τ entries and resolves sign conflicts before scaling; DARE (Yu et al., 2024) randomly drops τ elements with rescaling. Both were run at their standard settings on three representative pairs and did not beat vanilla summation by more than ± 2 pts.

Compute. Hardware: P6-B200 nodes (8× GPU). Framework: TRL 1.0.0, transformers 4.57.6, `reasoning-gym`. Approximate GPU-hours. Phase 1: 100; Phase 2: 80; Phase 3: 10 (analysis-only, CPU-bound SVDs); Phase 4 Parts 1–5: 250; Phase 4 Part 6: 180. **Total ≈600 GPU-hours.**

H Weight-Space Geometry: Metric Definitions

Full definitions of the six quantities introduced in §6. Throughout, W is a weight matrix with singular value decomposition $W = U\Sigma V^\top$ (U, V orthonormal, $\Sigma = \text{diag}(\sigma_1 \geq \cdots \geq \sigma_r)$); W_{SFT} and W_{GRPO} are the matrix before and after GRPO; $\tau = W_{\text{GRPO}} - W_{\text{SFT}}$; and U_k, V_k are the top-k left/right singular vectors. Every metric is computed per weight matrix, then averaged over tasks and aggregated by layer type or by depth (the pairwise novel-subspace angle is instead reported per task pair, as it has no single-task value).

Task vector and magnitude. $\tau = W_{\text{GRPO}} - W_{\text{SFT}}$ is GRPO's learned delta. Its relative size is $\|\tau\|_F / \|W\|_F$, where $\|M\|_F = \sqrt{\sum_{ij} M_{ij}^2} = \sqrt{\sum_i \sigma_i(M)^2}$ is the Frobenius norm.

Singular values and spectral norm. $\sigma_i(W)$ is the gain W applies along its i-th principal direction; $\sigma_1 = \|W\|_2$ (the spectral norm) is the largest amplification W can apply to any unit vector. The *spectral-norm ratio* $\sigma_1(W_{\text{GRPO}})/\sigma_1(W_{\text{SFT}})$ and the per-index relative shifts $|\sigma_i(W_{\text{GRPO}}) - \sigma_i(W_{\text{SFT}})|/\sigma_i(W_{\text{SFT}})$ test whether GRPO rescales the directions W already uses. A ratio of 1.000 means the dominant geometry of W is untouched.

Effective rank. $\mathrm{erank}(M) = \exp\big(H(\{\sigma_i/\sum_j \sigma_j\})\big)$, where H is the Shannon entropy of the normalized singular-value distribution. High effective rank means M's action is spread across many directions; low means it is concentrated in a few. $\Delta\mathrm{erank} = \mathrm{erank}(W_{\mathrm{GRPO}}) - \mathrm{erank}(W_{\mathrm{SFT}})$ tests whether GRPO concentrates or diffuses W's representations.

Subspace projection. $\rho_k(\tau) = \|U_k U_k^\top \tau V_k V_k^\top\|_F^2 / \|\tau\|_F^2$ is the fraction of the update's energy lying inside W's top-k singular subspace. For an isotropic random τ in an r-dimensional ambient space, $\mathbb{E}[\rho_k] = (k/r)^2$, the "no special alignment" null baseline. $\rho_k \approx (k/r)^2$ means τ is directionally random with respect to W (i.e. orthogonal to its dominant subspace); $\rho_k < (k/r)^2$ means τ *actively avoids* W's top directions.

Principal angles (basis rotation, magnitude). For two k-dimensional subspaces with orthonormal bases A, B, the principal angles are $\theta_i = \arccos \sigma_i(A^\top B)$. We report the mean of θ_i between U_{SFT} and U_{GRPO} (top-k, k=100): 0° means the basis is frozen, 90° means it is fully reoriented. This measures only *how much* the basis rotated, not *in which direction*.

Novel subspace and pairwise angle (rotation direction). The new directions task A adds beyond SFT are $\mathrm{novel}_A = (I - U_{\mathrm{SFT}} U_{\mathrm{SFT}}^\top)\, U_{\mathrm{GRPO}(A)}[:, :k]$. The principal angle between $\mathrm{span}(\mathrm{novel}_A)$ and $\mathrm{span}(\mathrm{novel}_B)$ measures whether two tasks rotate the *same* way (0°) or into perpendicular directions (90°). The *flat rotation cosine* $\cos\big(\mathrm{vec}\,\mathrm{novel}_A, \mathrm{vec}\,\mathrm{novel}_B\big)$ is a signed scalar view of the same alignment: +1 aligned, 0 orthogonal, −1 directly opposing (the microscopic signature of interference).

Depth signature. For each task, $\mathrm{depth}[\ell] = \sum_{\text{matrices at block } \ell} \|\tau\|_F$ is a length-L profile of change magnitude across transformer blocks (normalized to sum to one). The *depth-profile cosine* between two tasks measures whether they edit the same depths.

Layer types. *Feature projectors* (`mlp.up_proj`, `mlp.gate_proj`, `attn.q_proj`, `attn.k_proj`) choose what features to produce or attend to, and are the natural site of basis rotation. *Sinks* (`mlp.down_proj`, `attn.o_proj`) aggregate features back into the residual stream, so their dominant directions are stable.

Across-task spread of the universal metrics. The "how" metrics of §6 are reported as means over the tasks trained at each scale; Table 26 gives the across-task spread behind those means, computed from the per-task values. Each cell is the task-averaged value with the max−min range across tasks in parentheses. The spread is negligible at every scale, the basis rotation varies by $\leq 0.06°$ across tasks, never exceeding the 0.1° stated in the body, so the task-averaged and per-task pictures coincide; this near-zero across-task variation is precisely what licenses calling these metrics universal. Relative update magnitude $\|\tau\|/\|W\|$ is the loosest of the four (it varies by up to 0.03 percentage points at 7B) but is still small and roughly task-independent. The spectral-norm ratio and effective-rank change are tighter still: across the nine 8B tasks the ratio is 1.0002 (range $[0.9991, 1.0018]$, s.d. 0.0007) and $\Delta\mathrm{erank}$ is -0.002 (range $[-0.009, +0.002]$), both indistinguishable from "no change" for every individual task.

Table 26: **Across-task spread of the universal "how" metrics.** Each cell: task-averaged value (max−min range across the tasks at that scale). Basis rotation is the principal angle at k=100. The across-task range stays $\leq 0.06°$ for rotation and ≤ 0.034 pp for relative magnitude, i.e. the metrics are effectively identical for every task.

Metric	3B	4B	7B	8B
Basis rotation `mlp.up_proj` (°)	1.47 (0.01)	1.99 (0.02)	2.49 (0.04)	2.67 (0.06)
Basis rotation `attn.q_proj` (°)	1.65 (0.01)	1.81 (0.02)	2.04 (0.06)	2.07 (0.04)
Basis rotation `mlp.down_proj` (°)	0.01 (0.01)	0.02 (0.01)	0.00 (0.00)	0.00 (0.00)
Rel. magnitude $\|\tau\|/\|W\|$ (%)	0.071 (0.012)	0.066 (0.013)	0.117 (0.034)	0.059 (0.011)

Per-task task-vector magnitude and its own spectrum (4B). Table 27 gives the whole-model Frobenius magnitude $\|\tau\|_F$ of each task vector and, separately, the spectral structure of τ itself (its leading singular value $\sigma_1(\tau)$ and the σ_1/σ_{10} decay, averaged over `mlp.down_proj` layers). Two facts back the

universal law that magnitude is roughly task-independent (§9, law 5): every task that learns sits in a narrow $\|\tau\|_F \in [0.73, 1.19]$ band regardless of difficulty, and `cryptarithm` (no reward signal) is the sole zero, a clean negative control. Meanwhile τ is itself moderately low-rank (2.5–4.2× decay across its top ten singular values), so GRPO writes a focused, preferred-direction update rather than a diffuse one.

Table 27: **Per-task task-vector magnitude and spectrum (Qwen3-4B).** $\|\tau\|_F$ is the whole-model Frobenius norm of the task vector; $\sigma_1(\tau)$ and the σ_1/σ_{10} decay characterize τ's own (per-layer) spectrum. `cryptarithm`'s reward never fires, so its τ is exactly zero (negative control). Rows ordered by $\|\tau\|_F$.

Task	$\|\tau\|_F$	$\sigma_1(\tau)$	σ_1/σ_{10}
leg_counting	1.193	0.014	3.0×
knights_knaves	1.005	0.017	3.7×
maze	0.994	0.018	4.2×
shortest_path	0.975	0.014	3.7×
countdown	0.970	—	—
caesar_cipher	0.909	0.016	4.0×
basic_arithmetic	0.852	0.014	3.7×
syllogism	0.756	0.011	3.2×
mini_sudoku	0.733	0.006	2.5×
cryptarithm	0.000	0.000	—

Per-task basis rotation (the "universal budget," de-aggregated). Table 5 and Fig. 4c report the basis rotation *averaged over tasks*. Table 28 breaks that average back into its per-task constituents for the tasks that received a full subspace-rotation analysis (`countdown` is omitted at 4B, where its run was data-contaminated, and `cryptarithm` everywhere, since its reward never fires so $\tau = 0$ and the rotation is undefined). Reading down any layer-type column within a scale, the per-task values are indistinguishable, differing by $\leq 0.06°$ (consistent with the across-task ranges in Table 26): this is the concrete evidence behind calling the rotation *magnitude* universal. Every task spends the same per-layer "rotation budget" on each layer type (`mlp.down_proj` frozen, the feature projectors rotating 1.5°–2.7° and growing with scale); only the *direction* (Fig. 5a) and the *depth* (Fig. 8) of that rotation are task-specific.

Table 28: **Per-task basis rotation by layer type and scale** (principal angle at k=100, degrees), the per-task de-aggregation of Table 5. Within any (scale, layer type) block the value barely moves across tasks ($\leq 0.06°$ spread): the rotation magnitude is governed by layer type and scale, not by the task.

Scale	Task	`mlp.down_proj`	`mlp.up_proj`	`attn.q_proj`
3B	basic_arithmetic	0.02°	1.47°	1.65°
	caesar_cipher	0.01°	1.47°	1.65°
	countdown	0.01°	1.48°	1.64°
	syllogism	0.01°	1.47°	1.65°
4B	basic_arithmetic	0.02°	1.99°	1.81°
	caesar_cipher	0.01°	2.00°	1.80°
	syllogism	0.02°	1.98°	1.82°
7B	basic_arithmetic	0.00°	2.51°	2.06°
	caesar_cipher	0.00°	2.47°	2.02°
	countdown	0.00°	2.47°	2.05°
	syllogism	0.00°	2.50°	2.00°
8B	basic_arithmetic	0.00°	2.68°	2.06°
	caesar_cipher	0.00°	2.70°	2.05°
	countdown	0.00°	2.64°	2.09°
	syllogism	0.00°	2.67°	2.06°

Per-task depth migration across scale (all tasks). The body tracks depth migration for four representative tasks (Fig. 5c). Figure 8 and Table 29 extend it to every task with a clean GRPO run, computed identically: each task's top-20 most-changed weight matrices are bucketed by transformer-block index into four depth quartiles (early/mid-early/mid-late/late), and we report the band center of mass (0 = early … 3 = late). The within-family pattern of the body holds and sharpens. Within Qwen2.5 (3B→7B) eight of nine tasks deepen; only `maze` moves shallower. Within Qwen3 (4B→8B) the split is starker: only `basic_arithmetic` and `shortest_path` deepen, while `syllogism`, `leg_counting`, `knights_knaves`, `mini_sudoku`, and `caesar_cipher` all move shallower. The *direction* of depth movement is thus task- and family-dependent, reinforcing §6's caution that a depth-based interference or curriculum analysis calibrated on one model, or one family, will mispredict another.

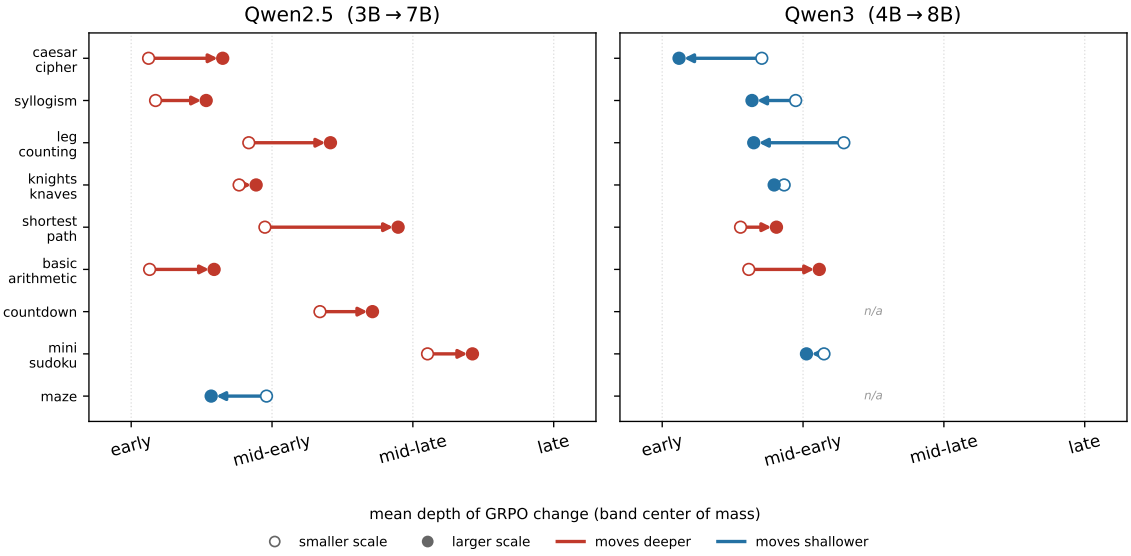


Figure 8: **Per-task depth migration, all tasks** (the all-task version of Fig. 5c). Each arrow runs from a family's smaller scale (open marker) to its larger scale (filled marker) at the task's depth center of mass; red = the change moves deeper with scale, blue = shallower. `countdown`@4B (contaminated) and `maze`@8B (not analyzed) are marked *n/a*. Qwen2.5 deepens almost universally (maze excepted); Qwen3 deepens only for `basic_arithmetic` and `shortest_path`.

Table 29: **Depth center of mass per task and scale** (band index, 0 = early … 3 = late; lower is shallower). These are the x-coordinates of Fig. 8. "–" marks an omitted run (`countdown`@4B contaminated, `maze`@8B not analyzed).

Task	3B	4B	7B	8B
caesar_cipher	0.12	0.71	0.65	0.12
syllogism	0.17	0.95	0.53	0.64
leg_counting	0.84	1.29	1.42	0.65
knights_knaves	0.77	0.87	0.89	0.80
shortest_path	0.95	0.56	1.90	0.81
basic_arithmetic	0.13	0.61	0.59	1.12
countdown	1.34	–	1.71	1.24
mini_sudoku	2.10	1.15	2.42	1.02
maze	0.96	0.21	0.57	–